

StructSynth: Dependency Structure Discovery for High-Fidelity Tabular Data Synthesis in Low-Data Regimes

Anonymous ACL submission

Abstract

The application of machine learning on tabular data in specialized domains is severely limited by data scarcity. Tabular data derives its value from the dependency relationships among features, yet reliably discovering these patterns becomes unstable when samples are scarce. Existing generative approaches either learn dependencies implicitly (failing in low-data regimes), assume pre-learned dependency graphs (which collapse under data scarcity), or induce dependencies from unstructured text (ignoring explicit structure). To address these limitations, we introduce **StructSynth**, a *discover-then-synthesize* framework that decouples dependency structure mining from data generation. In the first stage, **StructSynth** performs explicit dependency structure discovery, combining LLM-based reasoning with statistical association cues to construct a Directed Acyclic Graph (DAG) from limited samples. In the second stage, this learned structure serves as a blueprint for structure-conditioned generation, where data synthesis proceeds autoregressively following topological order, ensuring each feature is conditioned on its parent nodes. This design guarantees, by construction, that synthetic data respects the discovered dependencies. Extensive experiments demonstrate that **StructSynth** achieves state-of-the-art downstream utility while providing the best privacy-fidelity trade-off, proving especially effective in challenging low-data scenarios.

1 Introduction

Most real-world structured information can be naturally represented as tables, making tabular data a foundational abstraction for modern data analytics and machine learning (Jonker and Gomstyn, 2025). This abstraction underpins high-stakes applications across healthcare (Choi et al., 2017), finance (Rundo et al., 2019), and education (Luan and Tsai, 2021), valued not

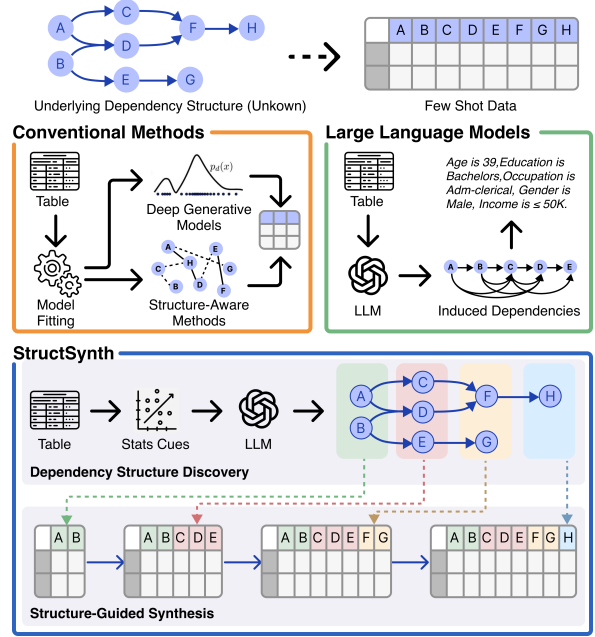


Figure 1: **StructSynth vs. Existing Paradigms: Decoupling Structure Discovery from Data Generation.** **Top:** Few-shot data encodes an unknown underlying dependency structure. **Middle:** Existing approaches either learn dependencies *implicitly* (deep generative models), assume *pre-learned* structure (structure-aware methods), or *induce* dependencies from linearized text (LLMs). **Bottom:** **StructSynth** explicitly discovers dependency structure via LLM-guided search with statistical cues, then synthesizes data layer-by-layer following topological order.

merely for its ubiquity but for the interpretable dependency relationships and statistical patterns it encodes. These dependencies—the probabilistic relationships among features—are what enable Machine Learning (ML) models to learn meaningful decision boundaries and make reliable predictions. However, in many critical domains such as rare medical conditions, emerging financial sectors, or underrepresented populations (Fonseca and Bacao, 2023), data scarcity poses a fundamental challenge. When the number of available samples

is severely limited ($n \leq 100$), three compounding failure modes emerge: (i) *statistical instability*—association tests (e.g., conditional independence tests, correlation estimates) lose statistical power, causing discovered dependencies to be dominated by spurious correlations rather than true structural signals; (ii) *structure learning collapse*—graph discovery algorithms that rely on these noisy statistics produce unreliable dependency structures, which in turn misguide any subsequent generation process; and (iii) *generative overfitting*—deep generative models, starved of training signal, tend to memorize the few available records rather than learning the underlying distribution, resulting in poor diversity and privacy violations. These failure modes are further compounded by common data characteristics including class imbalance, missing values, mixed categorical-continuous types, and long-tail category distributions. The resulting paradox—the widespread availability of the tabular format alongside the critical lack of sufficient samples in key areas—undermines the training, validation, and deployment of robust ML solutions (Chai et al., 2022; Borisov et al., 2022). Synthetic data generation offers a potential remedy, but its value lies not in merely augmenting volume; rather, effective synthesis must **preserve the dependency relationship** inherent in the original data, lest the augmentation introduce noise rather than signal.

To address data scarcity, conventional generative approaches have been developed with distinct strategies for handling feature dependencies (see Figure 1). **Deep Generative Models (DGMs)**, including Variational Autoencoders (Patki et al., 2016), Generative Adversarial Networks (Xu et al., 2019), and Diffusion Models (Kotelnikov et al., 2023), learn complex feature dependencies *implicitly* by fitting the data distribution. However, this implicit dependency learning process is severely compromised when training data is sparse, causing the learned patterns to distort and generating low-fidelity synthetic samples. **Structure-Aware Methods** take a different approach by explicitly modeling the data’s underlying relational structure as an inductive bias. This principle underlies foundational methods like Bayesian Networks (Ankan and Panda, 2015; Zhang et al., 2017) and has been integrated into advanced deep learning frameworks such as DECAF (Van Breugel et al., 2021) and GOGGLE (Liu et al., 2023). However, the robustness of these methods hinges on the accurate

discovery of the dependency graph. In data-scarce scenarios, the estimation of this graph becomes unstable, thereby compromising the integrity of the generation process. More recently, **Large Language Models (LLMs)** have emerged as a powerful paradigm for few-shot generation, leveraging vast prior knowledge to seemingly bypass data scarcity (Seedat et al., 2024). However, LLMs process tabular data through linearized textual representations, which means they can only *induce* dependencies sequentially from the input sequence rather than capturing explicit graphical relationships (Liu et al., 2024b). This structure-blind approach often produces synthetic data that fails to respect the true underlying dependency structure. In conclusion, existing paradigms either fit dependencies *implicitly* (data-hungry), assume reliable graph discovery (which collapses in low-data regimes), or *induce* dependencies from unstructured text (structure-blind).

These observations motivate a **discover-then-synthesize** paradigm that decomposes tabular data synthesis into two distinct stages: (1) *dependency structure discovery*—mining stable relational patterns from limited samples while avoiding spurious correlations, and (2) *structure-conditioned generation*—synthesizing data that explicitly respects the discovered structure rather than merely inducing it from sequential input. This framing leads to two fundamental research questions: (RQ1) *How can we reliably discover the underlying dependency structure when data is limited?* (RQ2) *How can we ensure that the generation process explicitly follows this discovered structure to achieve high structural fidelity?* Addressing these questions poses significant challenges: limited samples destabilize statistical association tests, making discovered dependencies prone to spurious correlations; and ensuring structural fidelity during generation requires more than sequential conditioning.

To address these challenges, we propose Struct-Synth, a decoupled two-stage framework for Structure-aware tabular data Synthesis, as illustrated in the lower panel of Figure 1. In the **Dependency Structure Discovery** stage, we propose an LLM-guided breadth-first search that integrates statistical association cues with the model’s prior knowledge to progressively construct a Directed Acyclic Graph (DAG)¹. A cycle resolu-

¹Here, we use “DAG” to denote a directed dependency

tion mechanism with explicit rationales ensures the graph remains acyclic while preserving the most plausible dependencies. In the **Structure-Guided Synthesis** stage, we leverage this DAG as a structural blueprint: nodes are partitioned into topological layers, and data synthesis proceeds autoregressively with each feature conditioned explicitly on its parent nodes. This design guarantees, *by construction*, that the synthetic data strictly adheres to the discovered dependency structure. Our contributions are summarized as follows:

- We propose a **discover-then-synthesize** framework that decouples dependency structure mining from data generation, enabling reliable pattern discovery under severe data scarcity.
- We introduce an LLM-guided structure discovery mechanism that combines statistical cues with prior knowledge and employs reasoned cycle resolution, paired with a **structure-conditioned generation** stage that enforces dependencies by design through topological layering and parent-conditioned synthesis.
- Extensive experiments demonstrate that Struct-Synth achieves state-of-the-art downstream utility (average rank 1.00 across six datasets) while providing the best **privacy-fidelity trade-off**, proving its effectiveness in challenging low-data regimes.

2 Related Works

2.1 Conventional Tabular Data Synthesis

Tabular data synthesis addresses data scarcity and privacy concerns by learning generative models to sample realistic data. Early approaches, such as the SMOTE (Chawla et al., 2002) and methods based on Copulas (Xue-Kun Song, 2000), were foundational but often struggled to capture complex, non-linear relationships.

Deep Generative Models (DGMs). This prompted a shift towards more expressive deep generative models (DGMs) that learn the data distribution implicitly. Advances in deep generative modeling, including Variational Autoencoders (VAEs) such as TVAE (Patki et al., 2016), Generative Adversarial Networks (GANs) exemplified by

CTGAN (Xu et al., 2019), and Normalizing Flow-based methods (Papamakarios et al., 2021), have significantly improved fidelity through implicit distribution learning. More recently, diffusion-based methods like TabDDPM (Kotelnikov et al., 2023) and TabSyn (Zhang et al., 2024) achieved state-of-the-art performance. TabSyn, in particular, encodes mixed-type tabular data into a continuous latent space via VAE and applies score-based diffusion therein, explicitly improving inter-column relation fidelity while reducing inference steps. However, since these DGMs implicitly learn complex feature dependencies, they still offer limited interpretability and controllability over the generation process.

Structure-Aware Methods. To enhance generative fidelity and controllability, structure-aware methods explicitly incorporate feature dependency structures into data synthesis. Early methods based on Bayesian Networks (Ankan and Panda, 2015) and PrivBayes (Zhang et al., 2017) generate samples guided by learned graphical structures, with PrivBayes introducing privacy-preserving mechanisms on top of the Bayesian framework. Recent deep-learning methods integrate structured inductive biases more effectively; DECAF (Van Breugel et al., 2021) leverages causal reasoning for fairness, while GOGGLE (Liu et al., 2023) employs a graph-based VAE. However, their effectiveness heavily depends on accurately learned graphs from data, posing challenges especially under limited data scenarios.

2.2 LLM-Based Tabular Data Synthesis

Large Language Models (LLMs), leveraging in-context learning and extensive prior knowledge, have recently advanced tabular data synthesis significantly. Fine-tuning approaches such as GRaT (Borisov et al., 2023), TAPTAP (Zhang et al., 2023), and AIGT (Zhang et al., 2025a) autoregressively generate data but incur high computational costs and deteriorate performance in low-data scenarios. Prompt-based methods, notably CuratedLLM (CLLM) (Seedat et al., 2024), alleviate these drawbacks by using curated examples to guide data generation without updating model parameters. More recently, TabGen-ICL (Fang et al., 2025) further advances the prompting paradigm by iteratively selecting in-context examples that represent the residual between the real and currently generated distributions, achieving progressive distribution alignment

graph: edges capture statistical/probabilistic dependencies inferred from limited observations and are used as a generative blueprint, not as a claim of ground-truth causality.

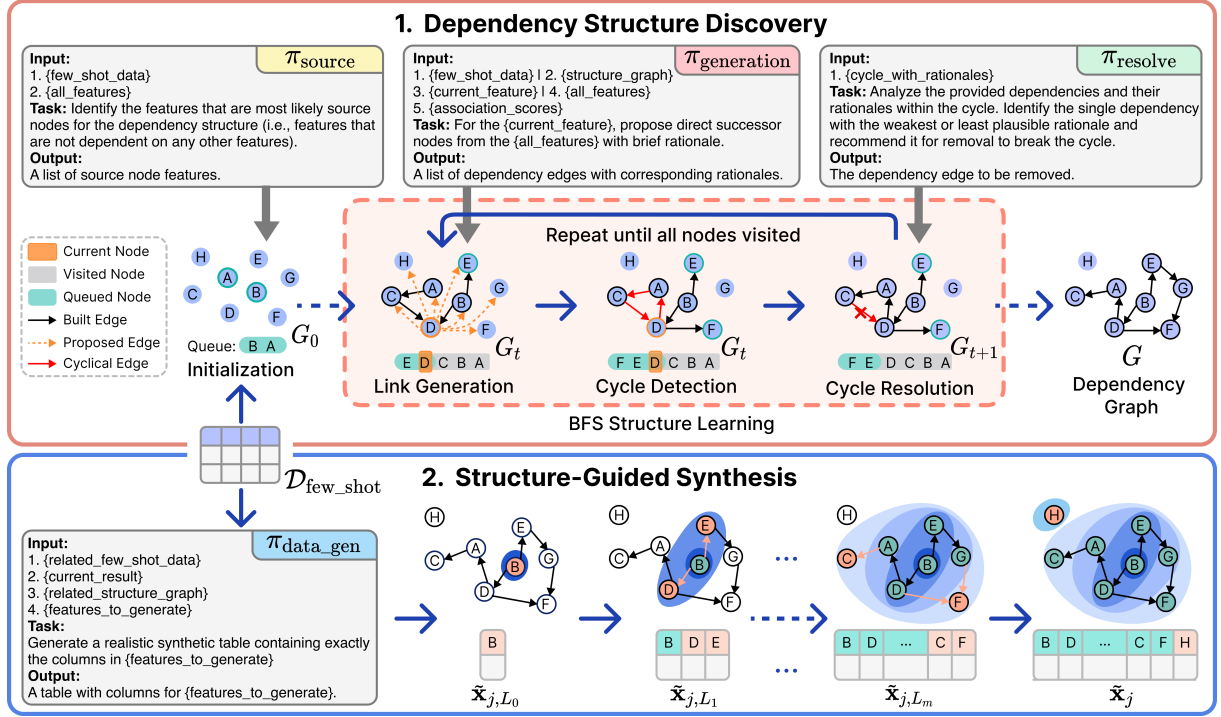


Figure 2: Overview of the StructSynth framework. The method first performs **Dependency Structure Discovery** to infer a dependency graph from limited data via an LLM-guided search. Subsequently, **Structure-Guided Synthesis** leverages the learned graph as a blueprint to autoregressively guide the LLM in generating new tabular data.

without any model fine-tuning. Beyond prompting strategies, GraDe (Zhang et al., 2025b) addresses the structural mismatch between dense Transformer attention and sparse tabular dependencies by injecting a learned sparse dependency graph into the attention mechanism, guided by externally extracted functional dependencies; this graph-guided attention yields notable improvements on structurally complex datasets. Nevertheless, while these methods induce or integrate dependencies implicitly or locally, they do not yield an interpretable global dependency topology that can serve as an explicit, executable blueprint for structure-preserving generation (Liu et al., 2024b).

3 Methodology

The proposed StructSynth leverages a Large Language Model (LLM) to perform efficient dependency graph discovery and subsequent data generation. As depicted in Figure 2, the methodology comprises two primary stages: a **structure learning phase** to discover the underlying dependency graph and a **graph-aware generation phase** that synthesizes new tabular data using the learned graph as a blueprint.²

²See Appendix I for full prompt templates with formatting details and Algorithm 1 in Appendix D for the complete

3.1 Problem Definition

Consider a small tabular training dataset, denoted as $\mathcal{D}_{\text{train}} = (\mathbf{X}_{\text{train}}, \mathbf{A})$. $\mathbf{X}_{\text{train}} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ represents a set of n instances (or rows), where the number of available samples is severely limited ($n \leq 100$). The set $\mathbf{A} = \{A_1, \dots, A_K\}$ defines the schema of the data through its K attributes (or columns). Our primary objective is to employ an LLM-based generation function, f_{LLM} , to produce a synthetic dataset, $\mathcal{D}_{\text{synth}} = (\mathbf{X}_{\text{synth}}, \mathbf{A})$, that is both realistic and diverse. The ultimate goal is to demonstrate that a downstream model trained on an augmented dataset, $\mathcal{D}_{\text{aug}} = (\mathbf{X}_{\text{train}} \cup \mathbf{X}_{\text{synth}}, \mathbf{A})$, exhibits superior performance on a held-out test set, $\mathcal{D}_{\text{test}}$, when compared to a model trained exclusively on the original, smaller dataset $\mathcal{D}_{\text{train}}$.

We adopt the DAG formulation for its functional role in defining a generative ordering; Appendix B provides a detailed justification.

3.2 Dependency Structure Discovery

The first step in StructSynth is to uncover the dependency structure present in the limited training data, represented as a DAG $G = (V, E)$. The node procedure.

set $V \subseteq \mathbf{A}$ consists of selected attributes from the dataset schema $\mathbf{A}$, while the edges E capture probabilistic dependencies among these attributes. The framework employs an iterative, breadth-first search (BFS) guided by an LLM to progressively construct the dependency graph. This design explicitly adapts to data-scarce settings by (a) leveraging the LLM’s semantic prior as a structural regularizer, (b) using statistical association scores as weak supervision, and (c) decoupling structure discovery from generation to reduce per-stage sample complexity. The core steps are described below.

3.2.1 Graph Initialization

We initiate the graph construction by identifying source nodes attributes that are not influenced by any others in the dataset. A small subset of the training data, $\mathcal{D}_{\text{few_shot}} \subseteq \mathcal{D}_{\text{train}}$, serves as in-context examples for the LLM, which is guided by a specifically designed prompt, π_{source} . Formally, the initial set of source nodes, V_{source} , is determined by the following operation:

$$V_{\text{source}} = f_{\text{LLM}}(\pi_{\text{source}}(\mathcal{D}_{\text{few_shot}})). \quad (1)$$

Consequently, the initial graph is set to $G_0 = (V_{\text{source}}, \emptyset)$. To manage the discovery process, a queue Q for the Breadth-First Search (BFS) is initialized with these source nodes, i.e., $Q \leftarrow V_{\text{source}}$, and a set for tracking explored attributes, V_{visited} , is initialized as empty.

3.2.2 Expansion and Reasoned Link Generation

The framework expands the graph G through an iterative, breadth-first process. In each iteration t , a node A_i is dequeued from the processing queue Q and added to the set of visited nodes, V_{visited} . To inform the model’s link generation with empirical data, we compute a vector of association scores, $\mathcal{S}(A_i) \in \mathbb{R}^{K-1}$, between A_i and all other attributes in $\mathbf{A}$. These scores are calculated using dependency measures specific to the data types of each attribute pair: *Pearson’s R* for continuous-continuous pairs, *Correlation Ratio* for categorical-continuous pairs, and *Cramér’s V* for categorical-categorical pairs (Agresti and Kateri, 2011) (see Appendix C for detailed formulas).

Subsequently, the LLM is queried with a comprehensive prompt, π_{generate} , which integrates the current graph state G_t , the node A_i , and the statistical scores $\mathcal{S}(A_i)$. The model’s task

is to propose a set of direct successors for A_i and provide a textual rationale for each proposed relationship. The structured output of this query is a set of successor-rationale pairs, denoted as P_i :

$$P_i = f_{\text{LLM}}(\pi_{\text{generate}}(A_i, G_t, \mathcal{S}(A_i), \mathcal{D}_{\text{few_shot}})). \quad (2)$$

From this set, we define the proposed new edges, $E_{\text{prop},t}$, and the set of newly discovered nodes, $V_{\text{new},t}$, which are not already in the graph’s vertex set V_t :

$$\begin{aligned} E_{\text{prop},t} &= \{(A_i \rightarrow A_j, r_{ij}) \mid (A_j, r_{ij}) \in P_i\}, \\ V_{\text{new},t} &= \{A_j \mid (A_j, _) \in P_i \wedge A_j \notin V_t\}. \end{aligned} \quad (3)$$

These new nodes are then added to the queue Q for subsequent exploration, ensuring the continued expansion of the graph.

3.2.3 LLM-based Cycle Resolution

To ensure that the graph remains a DAG throughout, the framework resolves any cycles that may result from integrating proposed edges $E_{\text{prop},t}$ into the current graph $G_t = (V_t, E_t)$. Specifically, we construct the candidate edge set $E_{\text{candidate}} = E_t \cup E_{\text{prop},t}$ and apply a cycle detection algorithm to identify all elementary cycles, Ψ .

If cycles are detected ($\Psi \neq \emptyset$), the framework initiates a reasoned resolution process. For each cycle $\psi \in \Psi$, where ψ represents the specific subset of edges from $E_{\text{candidate}}$ that forms the circular path (e.g., $\{(A \rightarrow B, r_{ab}), (B \rightarrow C, r_{bc}), (C \rightarrow A, r_{ca})\}$), the LLM is queried with the prompt π_{resolve} . The prompt’s input, ψ , provides the model with the complete logical contradiction, including the rationales for both newly proposed and previously established edges. The model’s task is to analyze this conflict and identify the single edge whose removal is most justified. This resolution logic is applied to every detected cycle. The complete set of pruned edges, E_{pruned} , is thus defined by collecting the output for each cycle in Ψ . The state for the next iteration, G_{t+1} , is then formally defined by removing these pruned edges from the candidate set:

$$\begin{aligned} E_{\text{pruned}} &= \{f_{\text{LLM}}(\pi_{\text{resolve}}(\psi)) \mid \psi \in \Psi\} \\ G_{t+1} &= (V_t \cup V_{\text{new},t}, (E_t \cup E_{\text{prop},t}) \setminus E_{\text{pruned}}). \end{aligned} \quad (4)$$

This general mechanism ensures the graph remains acyclic and empowers the model to revise its structure based on new, more confident evidence.

3.2.4 Continuation of BFS

The process continues iteratively. Following the cycle resolution step, the newly discovered nodes, $V_{\text{new},t}$, which are now part of the updated graph G_{t+1} , are added to the processing queue Q . The framework then returns to the *Expansion and Reasoned Link Generation* step to process the next node from the queue. This iterative expansion-resolution cycle continues until Q is empty, at which point the complete, acyclic dependency graph G is returned as the final output.

3.3 Structure-Guided Synthesis

Upon learning the dependency structure $G = (V, E)$, we utilize it to guide synthetic tabular data generation. Constructing each synthetic data point $\tilde{x}_j$ is performed in two sequential phases aligned with the underlying data structure.

3.3.1 Generating Graph-Based Values

This phase generates values for attributes V defined by the learned dependency graph. Initially, the nodes V are partitioned into topological layers $\mathcal{L} = (L_1, \dots, L_m)$. For each layer L_i , we extract the corresponding dependency subgraph $G_i = G[L_i \cup \Gamma^-(L_i)]$, where $\Gamma^-(L_i)$ represents the parent nodes of attributes in L_i . Additionally, we select the relevant slice of the few-shot dataset as $\mathcal{D}_{\text{few_shot}}^i = \Pi_{L_i \cup \Gamma^-(L_i)}(\mathcal{D}_{\text{few_shot}})$, where Π denotes the projection operator onto specified columns.

Values for attributes in layer L_i , denoted as $\tilde{x}_{j,L_i}$, are generated conditionally based on previously generated values for preceding layers $\tilde{x}_{j,<i}$:

$$\tilde{x}_{j,L_i} = f_{\text{LLM}}(\pi_{\text{data_gen}}(\tilde{x}_{j,<i}, G_i, L_i, \mathcal{D}_{\text{few_shot}}^i)) \quad (5)$$

Iterating through all layers yields a partial synthetic data point $\tilde{x}_{j,V}$ comprising values for all graph-dependent attributes.

3.3.2 Generating Independent Values

Subsequently, values for isolated attributes $A_{\text{iso}} = \mathbf{A} \setminus V$ are generated collectively in a single step, conditioned upon the complete set of previously generated graph-based values $\tilde{x}_{j,V}$:

$$\tilde{x}_{j,A_{\text{iso}}} = f_{\text{LLM}}(\pi_{\text{data_gen_iso}}(\tilde{x}_{j,V}, A_{\text{iso}}, \mathcal{D}_{\text{few_shot}}^{\text{iso}})) \quad (6)$$

where $\mathcal{D}_{\text{few_shot}}^{\text{iso}} = \Pi_{A_{\text{iso}}}(\mathcal{D}_{\text{few_shot}})$.

3.3.3 Final Assembly

A complete synthetic data point $\tilde{x}_j$ is formed by concatenating the graph-dependent and independent attribute values as $\tilde{x}_j = \text{concat}(\tilde{x}_{j,V}, \tilde{x}_{j,A_{\text{iso}}})$. Repeating this procedure s times yields the synthetic dataset $\mathbf{X}_{\text{synth}} = \{\tilde{x}_1, \dots, \tilde{x}_s\}$, resulting in the final synthetic dataset $\mathcal{D}_{\text{synth}} = (\mathbf{X}_{\text{synth}}, \mathbf{A})$.

4 Experiments

4.1 Experimental Setups

4.1.1 Datasets

We conduct experiments on six real-world tabular datasets: Adult (Becker and Kohavi, 1996), Anxiety (Henderson et al., 2025), Compas (Angwin et al., 2016), Salary (Sajjad, 2025), Obesity (Palechor and de la Hoz Manotas, 2019), and Churn (Keramati et al., 2014). These datasets cover a range of domains (e.g., social, medical, business) and downstream tasks including binary/multi-class classification and regression. Notably, Anxiety and Salary were released after the knowledge cutoff of modern LLMs, allowing us to assess performance in the absence of prior exposure. Full dataset details are provided in Appendix E.1. Additionally, for evaluating structure discovery quality, we utilize three benchmark datasets from bnlearn (Scutari, 2010) with known ground-truth DAGs: Asia (Scutari, 1996a) (8 nodes), Child (Scutari, 1996b) (20 nodes), and Insurance (Scutari, 1996c) (27 nodes). These allow direct measurement of structural recovery via the Structural Hamming Distance (SHD) (de Jongh and Druzdzel, 2009).

4.1.2 Baselines

We compare our method against eleven baselines from three categories (implementation details in Appendix E.2): 1) **Deep Generative Models (DGMs)**, including TVAE (Xu et al., 2019), CTGAN (Xu et al., 2019), TabDDPM (Kotelnikov et al., 2023), NFlow (Papamakarios et al., 2021), and TabSyn (Zhang et al., 2024); 2) **Structure-Aware Methods**, including Bayesian Networks (Ankan and Panda, 2015), GOGGLE (Liu et al., 2023), and DECAF (Van Breugel et al., 2021); and 3) **LLM-based Methods**, including GReaT (Borisov et al., 2023), CLLM (Seedat et al., 2024), and GraDe (Zhang et al., 2025b).

Table 1: Comparison of Models on Downstream Model Performance (%). (C) and (R) denote classification and regression tasks, respectively. [†] indicates datasets created post LLM knowledge cutoff. Best results are in **bold**; second-best are underlined.

Type	Method	Adult (C)	Anxiety (C) [†]	Compas (C)	Salary (R) [†]	Obesity (R)	Churn (C)	Average	Avg. Rank
Real	$\mathcal{D}_{\text{train}}$	82.51 $\pm$ 1.93	85.49 $\pm$ 1.60	64.84 $\pm$ 2.51	49.71 $\pm$ 7.02	59.83 $\pm$ 6.40	86.86 $\pm$ 3.78	71.54	-
DGMs	TVAE	81.21 $\pm$ 1.54	79.63 $\pm$ 2.65	64.39 $\pm$ 2.06	46.18 $\pm$ 5.91	60.92 $\pm$ 2.61	87.42 $\pm$ 3.80	69.96	6.50
	DDPM	81.72 $\pm$ 1.79	82.94 $\pm$ 3.24	66.01 $\pm$ 2.50	22.30 $\pm$ 15.70	43.20 $\pm$ 7.98	85.91 $\pm$ 2.05	63.68	8.17
	CTGAN	81.14 $\pm$ 1.81	76.06 $\pm$ 1.47	65.17 $\pm$ 2.61	46.17 $\pm$ 5.25	54.29 $\pm$ 5.90	86.50 $\pm$ 4.17	68.22	7.83
	NFlow	75.42 $\pm$ 4.58	74.83 $\pm$ 2.94	63.68 $\pm$ 4.22	25.55 $\pm$ 6.16	46.80 $\pm$ 4.17	84.99 $\pm$ 2.73	61.88	11.00
	TabSyn	83.88 $\pm$ 1.56	85.68 $\pm$ 2.40	67.77 $\pm$ 6.98	47.17 $\pm$ 9.58	43.50 $\pm$ 9.91	89.86 $\pm$ 1.53	69.64	3.83
Structure Aware	GOGGLE	78.92 $\pm$ 3.04	80.92 $\pm$ 1.61	63.73 $\pm$ 3.85	43.18 $\pm$ 10.65	55.84 $\pm$ 7.77	85.27 $\pm$ 3.98	67.98	8.50
	BN	82.68 $\pm$ 1.35	82.77 $\pm$ 2.39	60.13 $\pm$ 3.46	46.33 $\pm$ 7.29	59.76 $\pm$ 3.46	88.85 $\pm$ 3.02	70.09	6.00
	DECAF	77.64 $\pm$ 3.91	79.90 $\pm$ 2.30	63.90 $\pm$ 3.69	43.58 $\pm$ 6.25	52.88 $\pm$ 6.44	83.94 $\pm$ 2.97	66.97	9.17
LLMs	GReaT	82.55 $\pm$ 2.20	83.81 $\pm$ 2.55	64.02 $\pm$ 2.72	51.43 $\pm$ 3.31	47.03 $\pm$ 9.16	86.67 $\pm$ 3.20	69.25	6.00
	CLLM	83.95 $\pm$ 2.09	85.17 $\pm$ 0.94	66.24 $\pm$ 1.51	54.53 $\pm$ 0.20	60.43 $\pm$ 7.19	89.81 $\pm$ 2.78	<u>73.36</u>	<u>2.67</u>
	GraDe	83.72 $\pm$ 1.44	79.74 $\pm$ 2.01	66.15 $\pm$ 4.90	35.03 $\pm$ 8.08	22.92 $\pm$ 7.01	88.81 $\pm$ 2.51	62.73	7.33
Ours	StructSynth	85.55$\pm$1.08	86.45$\pm$0.57	69.40$\pm$0.25	55.98$\pm$2.96	62.27$\pm$5.40	90.39$\pm$1.96	75.01	1.00

4.1.3 Evaluation Metrics

Following previous methods (Borisov et al., 2023; Seedat et al., 2024), we evaluate the quality of synthetic data using three different metrics, with detailed definitions provided in Appendix E.4:

- **Downstream Model Performance:** This metric evaluates the data’s utility by training a downstream model on the augmented dataset $\mathcal{D}_{\text{aug}} = (\mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{synth}})$ and conducting an evaluation on the real test set $\mathcal{D}_{\text{test}}$. We report Area Under the Curve (AUC) for classification and R-squared (R^2) for regression.
- **Statistical Fidelity:** This metric evaluates how well the statistical properties of the synthetic dataset, $\mathcal{D}_{\text{synth}}$, replicate those of the original dataset, $\mathcal{D}_{\text{train}}$. We calculate it as the mean absolute difference between the pairwise correlation matrices of $\mathbf{X}_{\text{synth}}$ and $\mathbf{X}_{\text{train}}$, where a **lower score indicates higher fidelity**.
- **Privacy Risk Assessment:** This metric assesses privacy risk by measuring if records in $\mathcal{D}_{\text{synth}}$ are overly similar to the $\mathcal{D}_{\text{train}}$. For each instance in $\mathbf{X}_{\text{synth}}$, we find its nearest neighbor in the set of all real records ($\mathbf{X}_{\text{train}} \cup \mathbf{X}_{\text{test}}$). The score is the fraction of synthetic records whose nearest neighbor is from $\mathbf{X}_{\text{train}}$, where a **value approaching 0.5 indicates better privacy preservation** (van Breugel et al., 2023).

4.1.4 Implementation Details

We use gpt-4o-mini with a temperature of 0.9 for both the CLLM and StructSynth models.

Our experiments evaluate the low-data regime, generating 1000 synthetic samples ($s = 1000$) from 100 original samples ($n = 100$). Downstream performance is assessed using an XGBoost model. All experiments are repeated 10 times, and we report results as mean $\pm$ standard deviation to ensure robustness. Full hyperparameter configurations are provided in Appendix E.3.

4.2 Main Results

4.2.1 Downstream Model Performance

We evaluate the generated data on downstream tasks and, for comparison, also report results using the original limited training set $\mathcal{D}_{\text{train}}$. The results are summarized in Table 1. Our method, StructSynth, consistently outperforms all baselines across all datasets, achieving a perfect average rank of 1.00 and the highest average score (75.01). This provides a significant improvement over using only the original $\mathcal{D}_{\text{train}}$ (71.54) and surpasses the strongest competitor, CLLM (73.36), demonstrating that explicitly guiding an LLM with a structural blueprint yields substantial gains. Concurrently, the weaker performance of standard DGMs (e.g., TabSyn) and prior structure-aware models (e.g., GOGGLE) validates our core hypotheses: implicit learning is unreliable with sparse data, and generative performance is crippled if the initial structure discovery fails. These findings strongly confirm that StructSynth’s decoupled design, which unifies robust structure learning with LLM-based generation, is a more effective approach for high-fidelity data synthesis in low-

Table 2: Comparison of Models on Privacy Preservation and Statistical Fidelity (%). [†] indicates datasets created post LLM knowledge cutoff. Best results are in **bold**; second-best are underlined.

Type	Method	Adult	Anxiety [†]	Compas	Salary [†]	Obesity	Churn	Average	Avg. Rank
Results for Privacy Preservation . A value approaching 0.5 indicates better privacy preservation.									
DGMs	TVAE	50.34 $\pm$ 3.91	55.41 $\pm$ 4.26	64.50 $\pm$ 5.18	55.09 $\pm$ 4.38	52.93 $\pm$ 7.86	58.70 $\pm$ 4.75	56.16	4.33
	DDPM	99.95 $\pm$ 0.05	66.20 $\pm$ 13.17	67.18 $\pm$ 18.06	63.32 $\pm$ 19.52	71.36 $\pm$ 16.88	62.89 $\pm$ 24.91	71.82	9.67
	CTGAN	50.70 $\pm$ 3.64	56.64 $\pm$ 4.72	64.43 $\pm$ 4.22	53.59 $\pm$ 2.05	53.44 $\pm$ 3.89	57.84 $\pm$ 2.31	56.11	4.67
	NFlow	54.36 $\pm$ 1.92	53.52$\pm$3.08	62.57 $\pm$ 3.66	52.76 $\pm$ 2.34	51.74 $\pm$ 3.68	52.97 $\pm$ 3.33	54.65	3.50
	TabSyn	51.64 $\pm$ 2.67	70.53 $\pm$ 7.45	67.55 $\pm$ 5.40	54.66 $\pm$ 4.19	63.30 $\pm$ 7.19	59.38 $\pm$ 3.70	61.18	7.50
Structure Aware	GOGGLE	49.15 $\pm$ 7.97	43.62 $\pm$ 33.49	65.16 $\pm$ 17.25	30.94 $\pm$ 18.49	44.28 $\pm$ 23.83	61.87 $\pm$ 18.01	49.17	7.17
	BN	53.73 $\pm$ 2.36	72.98 $\pm$ 4.69	67.78 $\pm$ 3.63	61.50 $\pm$ 1.48	68.02 $\pm$ 2.18	59.68 $\pm$ 3.74	63.95	8.83
	DECAF	46.23 $\pm$ 19.13	58.59 $\pm$ 10.37	64.07 $\pm$ 19.14	56.91 $\pm$ 17.41	<u>50.82$\pm$15.49</u>	69.79 $\pm$ 18.81	57.74	6.33
LLMs	GReaT	99.50 $\pm$ 0.35	85.39 $\pm$ 2.29	94.10 $\pm$ 1.33	86.66 $\pm$ 2.45	80.75 $\pm$ 2.99	94.51 $\pm$ 1.48	90.15	11.83
	CLLM	49.45 $\pm$ 3.85	44.37 $\pm$ 4.55	64.14 $\pm$ 4.22	<u>50.96$\pm$5.40</u>	45.36 $\pm$ 2.69	48.78$\pm$5.82	50.51	<u>3.33</u>
	GraDe	97.51 $\pm$ 1.60	61.14 $\pm$ 1.94	75.11 $\pm$ 5.11	65.99 $\pm$ 2.60	59.30 $\pm$ 4.76	69.07 $\pm$ 2.86	71.35	9.50
Ours	StructSynth	49.97$\pm$5.78	<u>44.74$\pm$5.13</u>	62.37$\pm$3.86	50.13$\pm$4.89	49.80$\pm$6.31	<u>48.67$\pm$4.84</u>	50.95	1.33
Results for Statistical Fidelity . A lower score indicates higher fidelity.									
DGMs	TVAE	53.71 $\pm$ 1.80	46.16 $\pm$ 1.28	60.26 $\pm$ 1.09	66.93 $\pm$ 1.13	<u>58.41$\pm$0.52</u>	53.22 $\pm$ 1.54	58.11	5.17
	DDPM	65.26 $\pm$ 1.05	52.76 $\pm$ 1.22	64.94 $\pm$ 1.79	90.51 $\pm$ 0.33	64.44 $\pm$ 1.36	64.40 $\pm$ 2.90	67.05	10.00
	CTGAN	52.37 $\pm$ 1.56	47.14 $\pm$ 2.39	59.96 $\pm$ 0.90	58.57 $\pm$ 0.89	58.71 $\pm$ 0.57	<u>52.32$\pm$1.75</u>	54.85	<u>4.17</u>
	NFlow	50.68$\pm$2.71	<u>43.50$\pm$1.77</u>	60.62 $\pm$ 0.93	<u>52.38$\pm$1.10</u>	58.85 $\pm$ 0.98	53.98 $\pm$ 2.24	53.34	4.50
	TabSyn	49.17 $\pm$ 1.34	55.31 $\pm$ 1.31	57.23 $\pm$ 2.24	62.90 $\pm$ 3.01	58.08 $\pm$ 0.54	53.99 $\pm$ 2.08	56.11	4.75
Structure Aware	GOGGLE	68.32 $\pm$ 1.38	51.58 $\pm$ 7.27	66.19 $\pm$ 1.13	87.15 $\pm$ 1.65	63.80 $\pm$ 2.40	65.51 $\pm$ 2.16	67.09	10.00
	BN	82.68 $\pm$ 1.35	82.77 $\pm$ 2.39	60.13 $\pm$ 3.46	46.33$\pm$7.29	59.76 $\pm$ 3.46	88.85 $\pm$ 3.02	70.09	8.17
	DECAF	62.71 $\pm$ 3.21	48.53 $\pm$ 1.53	63.82 $\pm$ 1.34	84.70 $\pm$ 2.41	60.93 $\pm$ 2.21	60.22 $\pm$ 0.88	63.49	8.17
LLMs	GReaT	82.71 $\pm$ 1.35	40.10$\pm$1.64	52.17$\pm$1.18	58.75 $\pm$ 0.89	56.83$\pm$0.87	48.18$\pm$2.16	<u>56.46</u>	3.33
	CLLM	61.93 $\pm$ 4.43	57.64 $\pm$ 1.51	59.74 $\pm$ 2.33	78.60 $\pm$ 0.98	65.74 $\pm$ 1.42	62.20 $\pm$ 3.68	64.31	8.50
	GraDe	84.30 $\pm$ 0.86	42.59 $\pm$ 0.93	53.82 $\pm$ 2.62	55.85 $\pm$ 1.42	57.44 $\pm$ 1.43	53.66 $\pm$ 1.90	57.94	<u>4.17</u>
Ours	StructSynth	57.60 $\pm$ 3.13	57.86 $\pm$ 1.98	<u>57.23$\pm$1.69</u>	64.64 $\pm$ 0.96	61.14 $\pm$ 1.10	58.63 $\pm$ 2.66	59.52	7.08

data regimes.

4.2.2 Privacy Preservation and Statistical Fidelity

We evaluate each generative model on the competing objectives of statistical fidelity and privacy preservation to analyze this critical trade-off. Our analysis reveals a fundamental tension in this domain, where high statistical accuracy often results from data memorization. This is evident in our baseline comparisons (Table 2). For instance, GraDe achieves strong fidelity (avg. rank 4.17) at the cost of poor privacy preservation (avg. rank 9.50), indicating that its attention-based generation tends toward data memorization. Similarly, methods like NFlow and CTGAN rank highly on reproducing pairwise correlations but exhibit mediocre privacy and poor downstream utility, revealing their inability to robustly capture complex dependencies. At the other end of the spectrum, prompt-based models like CLLM, which do not directly model the training data distribution,

achieve strong privacy (avg. rank 3.33) but offer lower statistical fidelity (avg. rank 8.50).

In contrast, StructSynth resolves this trade-off more effectively, outperforming competitors like CLLM on both fronts. It achieves the best privacy preservation among all models (avg. rank 1.33) while also delivering competitive statistical fidelity (avg. rank 7.08). We attribute this performance to our decoupled framework, which uses an explicit structural blueprint as a regularizer to guide the LLM. This strategy constrains overfitting to individual training records, thereby enhancing privacy, while still capturing essential feature dependencies to ensure data utility. Overall, StructSynth provides a robust and reliable solution for synthesizing tabular data with a superior privacy-fidelity trade-off.

4.3 Ablation Study

To isolate the contributions of our method’s two main stages, we evaluate seven variants. For the structure learning stage: 1) *PC Discovery*,

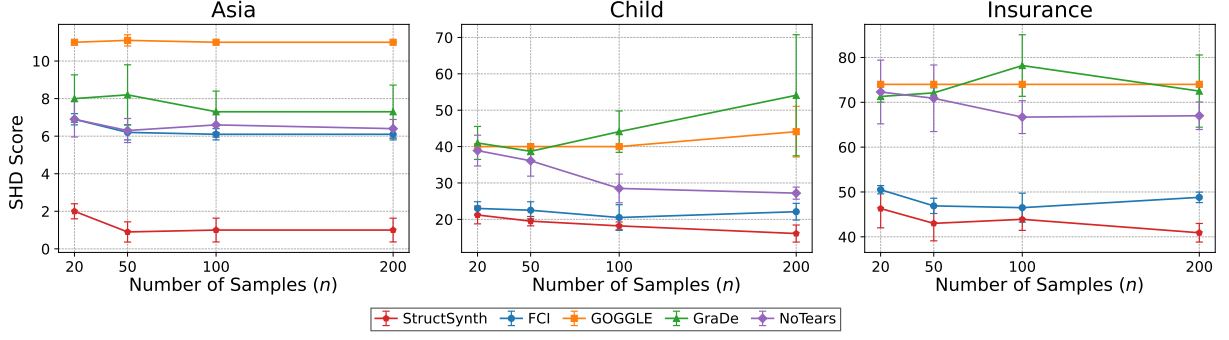


Figure 3: SHD of discovered dependency graphs vs. number of training samples on Asia, Child, and Insurance datasets with ground-truth DAGs. Lower SHD indicates better structural recovery.

Table 3: Ablation Study on Adult Datasets.

	Method	AUC	Fidelity	Privacy
	StructSynth	85.55	57.96	49.97
Structure Learning	PC Discovery	84.54	59.69	50.52
	NoTears Discovery	84.17	59.49	47.85
	No-Correlation Score	84.06	59.10	46.60
	Pairwise Discovery	85.25	57.37	49.17
Generation	No Topological Order	84.41	60.47	51.03
	No Structure	83.95	60.67	49.45
	Bayesian Sampler	81.17	48.86	50.02

substituting our LLM-guided discovery with the classical Peter–Clark algorithm (Spirtes et al., 2000); 2) *NoTears Discovery*, replacing our method with the continuous optimization-based NoTears algorithm (Zheng et al., 2018); 3) *No-Correlation Score*, removing statistical correlation scores from the prompt to examine their guiding influence; and 4) *Pairwise Discovery*, replacing the efficient BFS traversal with less scalable pairwise queries to evaluate the performance-efficiency trade-off. For the data generation stage: 5) *No Topological Order*, conducting data generation based on the complete learned structure but ignoring the topological order; 6) *No Structure*, removing structural guidance entirely (equivalent to the CLLM); and 7) *Bayesian Sampler*, substituting the LLM generator with a traditional Bayesian sampler fitted to the learned graph.

The results, summarized in Table 3, offer three key insights. First, **LLM-guided discovery outweighs classical baselines**: StructSynth outperforms both *PC* and *NoTears* (by 1.0–1.4 AUC pts). This suggests that leveraging semantic priors helps identify robust structures in low-data regimes where pure statistical methods often fail. Second, **structural guidance is multifaceted**: removing the graph (*No Structure*) or ignoring its hierarchy (*No Topological Order*) both degrade

utility (by 1.6 and 1.1 pts), proving that both explicit dependencies and their correct causal ordering are essential. Finally, **synergy is vital**: replacing the LLM with a *Bayesian Sampler* causes the sharpest drop (−4.4 pts), confirming that structural blueprints effectively guide—but cannot replace—the LLM’s superior distributional learning capabilities. Additionally, removing statistical prompts (*No-Correlation*) degrades performance, highlighting their value as weak supervision.

4.4 Structural Fidelity under Ground-Truth Graphs

To rigorously evaluate the quality of the discovered structures, we employ three benchmark datasets from the bnlearn repository—Asia, Child, and Insurance—which possess known ground-truth DAGs. This allows us to directly quantify structural fidelity using the Structural Hamming Distance (SHD), which is standard in the literature and measures the number of edge insertions, deletions, or reversals required to transform the learned graph into the ground truth. A lower SHD indicates a more accurate recovery of the underlying causal mechanisms.

Figure 3 presents the SHD across varying training sample sizes ($n \in \{20, 50, 100, 200\}$). StructSynth consistently achieves the lowest or near-lowest SHD across all datasets, demonstrating superior stability in low-data regimes. In contrast, traditional constraint-based methods like FCI and score-based methods like GOGGLE often yield significantly higher errors, particularly when n is small.

On the **Asia** dataset, which is small but prone to spurious correlations in low-sample settings, StructSynth maintains an SHD near 1, whereas baselines fluctuate at much higher error rates. This

suggests that our LLM-guided approach effectively utilizes prior knowledge to filter out false positives that purely statistical methods might mistake for true edges. For the **Child** dataset, which features a more complex structure, errors in early discovery stages can cascade. StructSynth’s iterative cycle resolution helps maintain graph validity, preventing the explosion of erroneous edges seen in methods like GraDe. Finally, on the **Insurance** dataset, which represents a medium-scale network with denser connections, our method again outperforms competitors. By prioritizing the discovery of a stable "structural blueprint" rather than attempting to model every minor correlation, StructSynth avoids the overfitting pitfalls that lead to the dense, incorrect graphs produced by some deep generative baselines. These results validate the first stage of our framework: StructSynth reliably identifies a high-fidelity dependency structure even from limited data, which is crucial for ensuring the subsequent generation follows valid causal pathways. We provide detailed examples on the Asia dataset in Appendix F.

4.5 Influence of Training Sample Size

To evaluate data efficiency, we varied the training size n from 20 to 200 on the Adult dataset and measured AUC, Statistical Fidelity, and Privacy Risk deviation Δ (Figure 4). Regarding AUC, StructSynth consistently outperforms all baselines. Notably, in the low-data regime ($n \leq 50$), it maintains a high baseline, whereas methods like BN and GOGGLE require significantly more samples to become competitive.

For Statistical Fidelity, StructSynth maintains a stable low score across all sample sizes. In contrast, baselines often fail to improve fidelity synchronously with AUC, drifting towards erroneous correlations. This confirms that our explicit structural blueprint aligns the generated distribution with real dependencies early on, minimizing statistical distortion even with limited data. Regarding privacy, we report the deviation $\Delta = |\text{PrivacyRisk} - 0.5|$. StructSynth’s Δ remains negligible across all n , indicating no tendency towards memorization. Conversely, baselines often exhibit larger Δ , reflecting the privacy-fidelity tension noted in Table 2. Our decoupled structure acts as a regularizer, preventing record-level overfitting. Overall, StructSynth exhibits a robust Pareto-like behavior: from low- ($n = 20$) to mid-data ($n = 200$) regimes, it simultaneously

achieves high AUC, low Fidelity scores, and minimal Privacy Δ , offering a stable trade-off independent of sample size.

4.6 Influence of Different Language Models

To assess the generalizability of StructSynth, we benchmark its performance against the CLLM baseline using a diverse set of LLMs on the Adult dataset. The evaluation spans a range of model sizes and includes both leading open-source models (Qwen-2.5 (Team, 2024), Llama-4 (AI, 2025), and DeepSeek (Liu et al., 2024a; Guo et al., 2025) series) and proprietary models (GPT-4o (Hurst et al., 2024) series), thereby highlighting the versatility of our approach. Experimental settings are kept consistent with those described in the main experiments, with the exception that, due to the high inference cost of DeepSeek R1, its evaluation is limited to 100 synthetic samples (i.e., $s = 100$).

The results, presented in Figure 5, reveal a consistent and robust performance advantage for StructSynth. Our method outperforms the CLLM across every tested architecture, from open-source models of varying sizes to powerful proprietary ones. Interestingly, the magnitude of this performance gain is most significant for models like Maverick (+0.028 AUC), demonstrating that our structural guidance can substantially boost the performance of models with moderate native capabilities. Even for top-tier models like GPT-4o, which are highly capable on their own, StructSynth still provides a measurable improvement. This overall pattern confirms that our framework offers a fundamental advantage, effectively elevating the performance ceiling regardless of the base LLM’s initial capabilities.

Appendix G provides a detailed token usage analysis showing that StructSynth incurs a modest +31% overhead over CLLM.

5 Conclusion

In this work, we introduced StructSynth, a two-stage framework that addresses the challenge of high-fidelity tabular data synthesis in low-data regimes through a *discover-then-synthesize* paradigm. The core insight is to decouple dependency structure discovery from data generation: the first stage constructs a Directed Acyclic Graph (DAG) via LLM-guided breadth-first search augmented with statistical association cues, serving as a generative blueprint rather than a causal

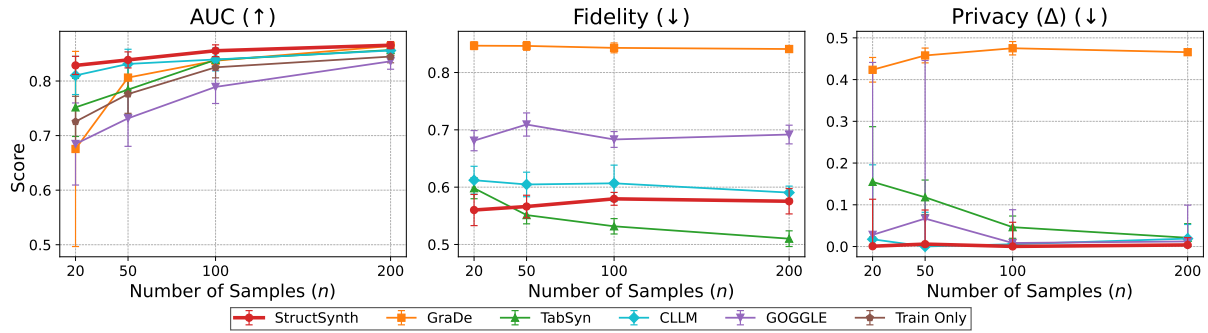


Figure 4: Effect of training sample size (n) on downstream utility (AUC $\uparrow$), statistical fidelity ($\downarrow$), and privacy risk deviation Δ ($\downarrow$) on the Adult dataset.

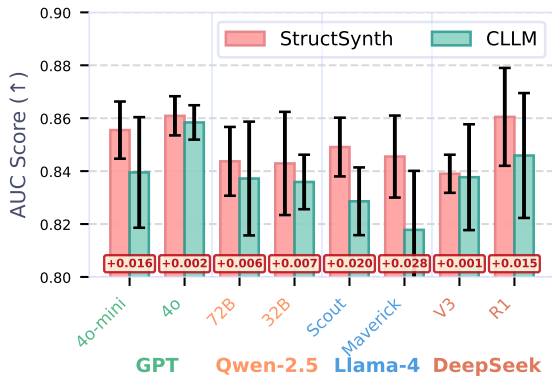


Figure 5: AUC Score on the Adult Dataset Using Various LLMs for StructSynth and CLLM.

Limitations

StructSynth relies on the LLM’s semantic prior over feature names and domains, so opaque or highly specialized schemas may weaken structure discovery. Its current evaluation covers moderate-dimensional tables ($K \in [8, 19]$); substantially wider schemas may require hierarchical discovery to control API cost and context length. The DAG formulation is functionally useful for autoregressive generation, but it cannot directly express cyclic or bidirectional dependencies. Finally, the current implementation depends on commercial LLM APIs, which introduces cost and reproducibility considerations despite the modest token overhead reported in Appendix G.

References

- Alan Agresti and Maria Kateri. 2011. Categorical data analysis. In *International encyclopedia of statistical science*, pages 206–208. Springer.
- Meta AI. 2025. [Introducing llama 4: Advancing multimodal intelligence](#). Meta AI blog.
- Julia Angwin, Jeff Larson, Lauren Kirchner, and Surya Mattu. 2016. Machine Bias — [propublica.org](https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing). <https://www.propublica.org/article/machine-bias-risk-assessments-in-criminal-sentencing>.
- Ankur Ankan and Abinash Panda. 2015. pgmpy: Probabilistic graphical models using python. In *SciPy*, pages 6–11.
- Alessandro Antonucci, Gregorio Piqu , and Marco Zaffalon. 2023. Zero-shot causal graph extrapolation from text via llms. *arXiv preprint arXiv:2312.14670*.
- Vahan Arsenyan, Spartak Bughdaryan, Fadi Shaya, Kent Wilson Small, and Davit Shahnazaryan. 2024. Large language models for biomedical knowledge graph construction: Information extraction from emr notes. In *Proceedings of the 23rd Workshop on*

claim; the second stage leverages this blueprint to synthesize data autoregressively through topological layering, guaranteeing adherence to feature dependencies by construction.

Extensive experiments across six real-world datasets demonstrate that StructSynth achieves state-of-the-art downstream utility with a perfect average rank of 1.00. Notably, it improves upon the strongest baseline (CLLM) by +1.65 AUC points on average, while simultaneously achieving the best privacy preservation (avg. rank 1.33) and maintaining competitive statistical fidelity. Our evaluations on three benchmark DAGs with known ground-truth structures confirm that the LLM-guided discovery reliably recovers dependency structures even from as few as 20 samples. Furthermore, the consistent performance gains across seven diverse LLM architectures—spanning both open-source and proprietary models—validate the generalizability of our structural guidance paradigm.

811	<i>Biomedical Natural Language Processing</i> , pages 295–	863
812	317.	864
813	Taiyu Ban, Lyuzhou Chen, Derui Lyu, Xiangyu Wang,	865
814	and Huanhuan Chen. 2023. Causal structure learning	866
815	supervised by large language model. <i>arXiv preprint</i>	867
816	<i>arXiv:2311.11689</i> .	868
817	Barry Becker and Ronny Kohavi. 1996. Adult.	869
818	UCI Machine Learning Repository. DOI:	870
819	https://doi.org/10.24432/C5XW20 .	871
820	Vadim Borisov, Tobias Leemann, Kathrin Seßler,	872
821	Johannes Haug, Martin Pawelczyk, and Gjergji	873
822	Kasneji. 2022. Deep neural networks and tabular	874
823	data: A survey. <i>IEEE transactions on neural</i>	875
824	<i>networks and learning systems</i> , 35(6):7499–7519.	876
825	Vadim Borisov, Kathrin Sessler, Tobias Leemann,	877
826	Martin Pawelczyk, and Gjergji Kasneji. 2023.	878
827	Language models are realistic tabular data generators .	879
828	In <i>The Eleventh International Conference on</i>	880
829	<i>Learning Representations</i> .	881
830	Chengliang Chai, Jiayi Wang, Yuyu Luo, Zeping Niu,	882
831	and Guoliang Li. 2022. Data management for	883
832	machine learning: A survey. <i>IEEE Transactions on</i>	884
833	<i>Knowledge and Data Engineering</i> , 35(5):4646–4667.	885
834	Nitesh V Chawla, Kevin W Bowyer, Lawrence O	886
835	Hall, and W Philip Kegelmeyer. 2002. Smote:	887
836	synthetic minority over-sampling technique. <i>Journal</i>	888
837	<i>of artificial intelligence research</i> , 16:321–357.	889
838	David Maxwell Chickering. 2002. Optimal structure	890
839	identification with greedy search. <i>Journal of machine</i>	891
840	<i>learning research</i> , 3(Nov):507–554.	892
841	Edward Choi, Siddharth Biswal, Bradley Malin, Jon	893
842	Duke, Walter F Stewart, and Jimeng Sun. 2017.	894
843	Generating multi-label discrete patient records using	895
844	generative adversarial networks. In <i>Machine</i>	896
845	<i>learning for healthcare conference</i> , pages 286–305.	897
846	PMLR.	898
847	Martijn de Jongh and Marek J Druzdzel. 2009. A	899
848	comparison of structural distance measures for	900
849	causal bayesian network models. <i>Recent advances</i>	901
850	<i>in intelligent information systems, challenging</i>	902
851	<i>problems of science, computer science series</i> , pages	903
852	443–456.	904
853	Liancheng Fang, Aiwei Liu, Hengrui Zhang,	905
854	Henry Peng Zou, Weizhi Zhang, and Philip S.	906
855	Yu. 2025. TABGEN-ICL: Residual-aware in-context	907
856	example selection for tabular data generation . In	908
857	<i>Findings of the Association for Computational</i>	909
858	<i>Linguistics: ACL 2025</i> , pages 20027–20041, Vienna,	910
859	Austria. Association for Computational Linguistics.	911
860	Joao Fonseca and Fernando Bacao. 2023. Tabular and	912
861	latent space synthetic data generation: a literature	913
862	review. <i>Journal of Big Data</i> , 10(1):115.	914
	Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song,	915
	Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong	916
	Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025.	917
	Deepseek-r1: Incentivizing reasoning capability in	918
	llms via reinforcement learning. <i>arXiv preprint</i>	
	<i>arXiv:2501.12948</i> .	
	Liam Henderson, Ethan Callahan, and Charlotte	
	Fraser. 2025. Social Anxiety Dataset — kag-	
	gle.com. https://www.kaggle.com/datasets/	
	natezhang123/social-anxiety-dataset .	
	Patrik Hoyer, Dominik Janzing, Joris M Mooij, Jonas	
	Peters, and Bernhard Schölkopf. 2008. Nonlinear	
	causal discovery with additive noise models. <i>Ad-</i>	
	<i>vances in neural information processing systems</i> , 21.	
	Aaron Hurst, Adam Lerer, Adam P Goucher, Adam	
	Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow,	
	Akila Welihinda, Alan Hayes, Alec Radford, and 1	
	others. 2024. Gpt-4o system card. <i>arXiv preprint</i>	
	<i>arXiv:2410.21276</i> .	
	Thomas Jiralerspong, Xiaoyin Chen, Yash More, Vedant	
	Shah, and Yoshua Bengio. 2024. Efficient causal	
	graph discovery using large language models. <i>arXiv</i>	
	<i>preprint arXiv:2402.01207</i> .	
	Alexandra Jonker and Alice Gomstyn. 2025. Struc-	
	tured vs. unstructured data: What’s the differ-	
	ence? https://www.ibm.com/think/topics/	
	structured-vs-unstructured-data . IBM Think.	
	Published Feb. 7, 2025. Accessed 2026-02-01.	
	A. Keramati, R. Jafari-Marandi, M. Aliannejadi,	
	I. Ahmadian, M. Mozaffari, and U. Abbasi. 2014.	
	Improved churn prediction in telecommunication	
	industry using data mining techniques . <i>Applied Soft</i>	
	<i>Computing</i> , 24:994–1012.	
	Emre Kiciman, Robert Ness, Amit Sharma, and	
	Chenhao Tan. 2023. Causal reasoning and large	
	language models: Opening a new frontier for	
	causality. <i>Transactions on Machine Learning</i>	
	<i>Research</i> .	
	Daphne Koller and Nir Friedman. 2009. <i>Probabilistic</i>	
	<i>Graphical Models: Principles and Techniques</i> . MIT	
	Press.	
	Akim Kotelnikov, Dmitry Baranchuk, Ivan Rubachev,	
	and Artem Babenko. 2023. Tabddpm: Modelling	
	tabular data with diffusion models. In <i>International</i>	
	<i>Conference on Machine Learning</i> , pages 17564–	
	17579. PMLR.	
	Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang,	
	Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi	
	Deng, Chenyu Zhang, Chong Ruan, and 1 others.	
	2024a. Deepseek-v3 technical report. <i>arXiv preprint</i>	
	<i>arXiv:2412.19437</i> .	
	Tennison Liu, Zhaozhi Qian, Jeroen Berrevoets, and	
	Mihaela van der Schaar. 2023. Goggle: Generative	
	modelling for tabular data by learning relational	
	structure. In <i>The Eleventh International Conference</i>	
	<i>on Learning Representations</i> .	

919	Tianyang Liu, Fei Wang, and Muhao Chen. 2024b.	Marco Scutari. 1996c. Bayesian network repository: Insurance.	974
920	Rethinking tabular data understanding with large		975
921	language models. In <i>Proceedings of the 2024</i>		
922	<i>Conference of the North American Chapter of the</i>	Marco Scutari. 2010. Learning bayesian networks with	976
923	<i>Association for Computational Linguistics: Human</i>	the bnlearn r package. <i>Journal of statistical software</i> ,	977
924	<i>Language Technologies (Volume 1: Long Papers)</i> ,	35:1–22.	978
925	pages 450–482.		
926	Hui Luan and Chin-Chung Tsai. 2021. A review of	Nabeel Seedat, Nicolas Huynh, Boris van Breugel,	979
927	using machine learning approaches for precision	and Mihaela van der Schaar. 2024. Curated LLM:	980
928	education. <i>Educational Technology & Society</i> ,	Synergy of LLMs and data curation for tabular	981
929	24(1):250–266.	augmentation in low-data regimes. In <i>Forty-first</i>	982
		<i>International Conference on Machine Learning</i> .	983
930	Benjamin S Manning, Kehang Zhu, and John Horton.	Shohei Shimizu, Patrik O Hoyer, Aapo Hyvärinen, Antti	984
931	2024. Automated social science: Language models	Kerminen, and Michael Jordan. 2006. A linear non-	985
932	as scientist and subjects. Technical report, National	Gaussian acyclic model for causal discovery. <i>Journal</i>	986
933	Bureau of Economic Research, Inc.	of <i>Machine Learning Research</i> , 7(10).	987
934	Narmada Naik, Ayush Khandelwal, Mohit Joshi,	Alik Sokolov, Fabrizio Sabelli, Behzad Azadie Faraz,	988
935	Madhusudan Atre, Hollis Wright, Kavya Kannan,	Wuding Li, and Luis Seco. 2024. Toward Automating	989
936	Scott Hill, Giridhar Mamidipudi, Ganapati Srinivasa,	Causal Discovery in Financial Markets and Beyond ,	990
937	Carlo Bifulco, and 1 others. 2024. Applying large	chapter CHAPTER 14.	991
938	language models for causal structure learning in		
939	non small cell lung cancer. In <i>2024 IEEE 12th</i>	Peter Spirtes, Clark N Glymour, Richard Scheines, and	992
940	<i>International Conference on Healthcare Informatics</i>	David Heckerman. 2000. <i>Causation, prediction, and</i>	993
941	(<i>ICHI</i>), pages 688–693. IEEE.	<i>search</i> . MIT press.	994
942	Fabio Mendoza Palechor and Alexis de la Hoz Manotas.	Peter Spirtes, Christopher Meek, and Thomas Richard-	995
943	2019. Dataset for estimation of obesity levels based	son. 1995. Causal inference in the presence of	996
944	on eating habits and physical condition in individuals	latent variables and selection bias. In <i>Proceedings of</i>	997
945	from colombia, peru and mexico . <i>Data in Brief</i> ,	<i>the Eleventh conference on Uncertainty in artificial</i>	998
946	25:104344.	<i>intelligence</i> , pages 499–506.	999
947	George Papamakarios, Eric Nalisnick, Danilo Jimenez	Qwen Team. 2024. Qwen2 technical report. <i>arXiv</i>	1000
948	Rezende, Shakir Mohamed, and Balaji Lakshmi-	<i>preprint arXiv:2407.10671</i> .	1001
949	narayanan. 2021. Normalizing flows for probabilistic		
950	modeling and inference. <i>Journal of Machine</i>	Boris Van Breugel, Trent Kyono, Jeroen Berrevoets, and	1002
951	<i>Learning Research</i> , 22(57):1–64.	Mihaela Van der Schaar. 2021. Decaf: Generating	1003
952	Neha Patki, Roy Wedge, and Kalyan Veeramachaneni.	fair synthetic data using causally-aware generative	1004
953	2016. The synthetic data vault. In <i>2016</i>	networks. <i>Advances in Neural Information</i>	1005
954	<i>IEEE international conference on data science and</i>	<i>Processing Systems</i> , 34:22221–22233.	1006
955	<i>advanced analytics (DSAA)</i> , pages 399–410. IEEE.		
956	Judea Pearl. 2009. <i>Causality: Models, Reasoning, and</i>	Boris van Breugel, Hao Sun, Zhaozhi Qian, and Mihaela	1007
957	<i>Inference</i> , 2 edition. Cambridge University Press.	van der Schaar. 2023. Membership inference attacks	1008
958	Zhaozhi Qian, Bogdan-Constantin Cebere, and Mihaela	against synthetic data through overfitting detection.	1009
959	van der Schaar. 2023. Synthcity: facilitating	In <i>International Conference on Artificial Intelligence</i>	1010
960	innovative use cases of synthetic data in different	<i>and Statistics</i> , pages 3493–3514. PMLR.	1011
961	data modalities .		
962	Francesco Rundo, Francesca Trenta, Agatino Luigi	Feng Xie, Ruichu Cai, Biwei Huang, Clark Glymour,	1012
963	Di Stallo, and Sebastiano Battiato. 2019. Machine	Zhifeng Hao, and Kun Zhang. 2020. Generalized	1013
964	learning for quantitative finance applications: A	independent noise condition for estimating latent	1014
965	survey. <i>Applied Sciences</i> , 9(24):5574.	variable causal graphs. In <i>NeurIPS</i> .	1015
966	Bisma Sajjad. 2025. Global AI Job Market &	Lei Xu, Maria Skoularidou, Alfredo Cuesta-Infante, and	1016
967	Salary Trends 2025 — kaggle.com. https:	Kalyan Veeramachaneni. 2019. Modeling tabular	1017
968	//www.kaggle.com/datasets/bismasajjad/	data using conditional gan. <i>Advances in neural</i>	1018
969	global-ai-job-market-and-salary-trends-2025 .	<i>information processing systems</i> , 32.	1019
970	Marco Scutari. 1996a. Bayesian network repository:	Peter Xue-Kun Song. 2000. Multivariate dispersion	1020
971	Asia (lung cancer) .	models generated from gaussian copula. <i>Scandina-</i>	1021
972	Marco Scutari. 1996b. Bayesian network repository:	<i>vian Journal of Statistics</i> , 27(2):305–320.	1022
973	Child .	Khadija Zanna and Akane Sano. 2025. Fairness-driven	1023
		llm-based causal discovery with active learning and	1024
		dynamic scoring. <i>arXiv preprint arXiv:2503.17569</i> .	1025

Hengrui Zhang, Jiani Zhang, Balasubramaniam Srinivasan, Zhengyuan Shen, Xiao Qin, Christos Faloutsos, Huzefa Rangwala, and George Karypis. 2024. Mixed-type tabular data synthesis with score-based diffusion in latent space. In *The twelfth International Conference on Learning Representations*.

Jun Zhang, Graham Cormode, Cecilia M Procopiuc, Divyesh Srivastava, and Xiaokui Xiao. 2017. Privbayes: Private data release via bayesian networks. *ACM Transactions on Database Systems (TODS)*, 42(4):1–41.

K Zhang and A Hyvärinen. 2009. On the identifiability of the post-nonlinear causal model. In *25th Conference on Uncertainty in Artificial Intelligence (UAI 2009)*, pages 647–655. AUAI Press.

Mingming Zhang, Zhiqing Xiao, Guoshan Lu, Sai Wu, Weiqiang Wang, Xing Fu, Can Yi, and Junbo Zhao. 2025a. Aigt: Ai generative table based on prompt. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 9926–9938.

Tianping Zhang, Shaowen Wang, Shuicheng Yan, Li Jian, and Qian Liu. 2023. Generative table pre-training empowers models for tabular prediction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14836–14854.

Zheyu Zhang, Shuo Yang, Bardh Prenkaj, and Gjergji Kasneci. 2025b. [Not all features deserve attention: Graph-guided dependency learning for tabular data generation with language models](#). In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 6217–6242, Suzhou, China. Association for Computational Linguistics.

Xun Zheng, Bryon Aragam, Pradeep K Ravikumar, and Eric P Xing. 2018. Dags with no tears: Continuous optimization for structure learning. *Advances in neural information processing systems*, 31.

Yujia Zheng, Biwei Huang, Wei Chen, Joseph Ramsey, Mingming Gong, Ruichu Cai, Shohei Shimizu, Peter Spirtes, and Kun Zhang. 2024. Causal-learn: Causal discovery in python. *Journal of Machine Learning Research*, 25(60):1–8.

A Structure Learning Background

A.1 Conventional Structure Learning

Conventional approaches to dependency structure discovery primarily fall into two categories: constraint-based and score-based methods. Constraint-based methods, such as the Peter–Clark (PC) algorithm (Spirtes et al., 2000) and the Fast Causal Inference (FCI) algorithm (Spirtes et al., 1995), utilize conditional independence tests to iteratively prune edges from an initially fully connected graph. While PC assumes causal sufficiency (no hidden confounders), FCI is specifically designed to handle latent variables. In contrast, score-based methods optimize a global scoring criterion to identify the most suitable graph structure. A representative example is the Greedy Equivalence Search (GES) (Chickering, 2002), which searches through equivalence classes to maximize a scoring function. However, both constraint-based and score-based approaches typically yield only Markov equivalence classes, thereby failing to distinguish between structurally distinct graphs that encode identical conditional independencies. To address this limitation, Functional Causal Models (FCMs) impose stronger assumptions regarding the data generation process. FCMs model effects explicitly as functions of direct causes combined with independent noise terms, enabling the identification of causal directions since independence between cause and noise typically breaks down when reversed (Shimizu et al., 2006; Hoyer et al., 2008; Zhang and Hyvärinen, 2009). Recent advancements have generalized this approach, introducing conditions such as Generalized Independent Noise (GIN) to manage latent causal variables in linear, non-Gaussian settings (Xie et al., 2020). However, the statistical power of these traditional algorithms heavily relies on a large number of samples, making them unreliable for structure discovery in the low-data regimes addressed by our work.

A.2 LLM-based Structure Learning

Leveraging Large Language Models (LLMs) for structure learning has recently emerged as a promising direction due to their inherent reasoning capabilities. Several paradigms have been proposed for integrating LLMs into structure discovery. Initially, methods focused on querying LLMs about pairwise causal relationships, showing

strong accuracy in determining causal directions between variable pairs (Kiciman et al., 2023). To overcome the quadratic complexity inherent to pairwise approaches, more scalable methods employing breadth-first search (BFS) strategies have been introduced, reframing structure learning as a graph traversal task and significantly reducing the required number of queries to the LLM (Jiralerspong et al., 2024; Zanna and Sano, 2025). A third paradigm enhances reliability by integrating an iterative supervision loop, in which traditional structure-learning algorithms propose initial graphs that are subsequently validated or corrected by LLMs. This method grounds LLM-generated structures in empirical evidence, thereby reducing hallucination risks (Ban et al., 2023). LLM-based structure discovery methods have demonstrated notable effectiveness across diverse domains, including medicine (Naik et al., 2024; Antonucci et al., 2023; Arsenyan et al., 2024), economics (Sokolov et al., 2024), and social sciences (Manning et al., 2024). Inspired by the success of LLMs in leveraging prior knowledge for domain-specific reasoning, our work integrates a scalable, LLM-driven paradigm for robust structure discovery.

B Structural Formulation: Why a DAG?

A central design choice in StructSynth is the representation of inter-feature dependencies as a Directed Acyclic Graph (DAG). Since tabular feature relationships are often associative rather than inherently directional, this choice warrants explicit justification. We adopt the DAG formulation for three principled reasons.

Pragmatic Blueprint, Not Causal Claim. We emphasize that our DAG is a *generative blueprint*: its edges encode statistical and probabilistic dependencies inferred from limited observations, not claims of ground-truth causality. This pragmatic interpretation aligns with the foundational Bayesian Network (BN) literature (Pearl, 2009; Koller and Friedman, 2009), where DAGs serve as compact encodings of conditional independence relationships without necessarily asserting causal direction. The distinction is important: StructSynth uses the discovered DAG to *organize* the generation process, not to make ontological claims about the data-generating mechanism.

Functional Necessity of Directionality. The structure-guided synthesis stage (Section 3.3) generates feature values autoregressively: each feature is produced conditioned on values already generated for its parent nodes. This autoregressive process inherently requires a *total ordering* over features to determine which values are available as conditioning context at each step. An undirected graph $\{A-B, B-C\}$ is ambiguous—it cannot specify whether A should be generated before C or vice versa. A DAG $\{A \rightarrow B \rightarrow C\}$ resolves this ambiguity by defining a topological order that directly translates into a layer-by-layer generation schedule. Even when the underlying statistical relationship between two features is symmetric (e.g., Age $\leftrightarrow$ Income), the generation process *must* select a direction—the DAG provides exactly this. The specific direction chosen does not affect *whether* the dependency is captured, only the *order* in which values are conditioned.

Acyclicity as a Mathematical Prerequisite. Beyond directionality, acyclicity is essential for the generation process to be well-defined. If cycles were permitted—e.g., $A \rightarrow B \rightarrow C \rightarrow A$ —the generation would encounter circular dependencies: the value of A would depend on C , which depends on B , which in turn depends on A , creating an unresolvable loop. The DAG constraint guarantees the existence of a valid topological ordering over nodes, ensuring that every feature can be generated exactly once, conditioned only on features whose values have already been determined.

Summary. In summary, the DAG in StructSynth serves a *functional* rather than *ontological* role: it imposes a well-defined, acyclic generation order that enables structure-conditioned autoregressive synthesis. This formulation is consistent with decades of probabilistic graphical model research and is a natural fit for our discover-then-synthesize paradigm.

C Statistical Association Measures

To quantify the empirical dependencies between attribute pairs, the framework employs three association measures, chosen according to the data types of the variables involved.

C.1 Continuous-Continuous: Pearson’s R

For two continuous variables $x = \{x_i\}_{i=1}^n$ and $y = \{y_i\}_{i=1}^n$, Pearson’s correlation coefficient r is

defined as the standardized covariance:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (7)$$

where $\bar{x}$ and $\bar{y}$ denote the sample means. The value of r ranges from $[-1, 1]$, where the sign indicates the direction of the linear relationship and the absolute value indicates its strength.

C.2 Categorical-Continuous: Correlation Ratio

The Correlation Ratio, η , measures the relationship between a categorical variable x and a continuous variable y . It is defined as the square root of the weighted variance of the category means divided by the total variance:

$$\eta = \sqrt{\frac{\sum_x n_x (\bar{y}_x - \bar{y})^2}{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (8)$$

where n_x is the number of samples in category x , $\bar{y}_x$ is the mean of y for samples in category x , and $\bar{y}$ is the overall mean. The value of η ranges from $[0, 1]$, with values closer to 1 indicating that the categorical variable provides more information about the continuous variable.

C.3 Categorical-Categorical: Cramér's V

For two categorical variables with a contingency table of dimensions $r \times k$, Cramér's V is based on the χ^2 statistic. To ensure robustness, we employ the bias correction proposed by Bergsma and Wicher (2013).

First, the bias-corrected ϕ^2 is calculated as:

$$\phi_{\text{corr}}^2 = \max\left(0, \frac{\chi^2}{n} - \frac{(k-1)(r-1)}{n-1}\right) \quad (9)$$

where n is the total number of samples. The corrected dimensions are:

$$r_{\text{corr}} = r - \frac{(r-1)^2}{n-1}, \quad k_{\text{corr}} = k - \frac{(k-1)^2}{n-1} \quad (10)$$

The final Cramér's V score is then given by:

$$V_{\text{corr}} = \sqrt{\frac{\phi_{\text{corr}}^2}{\min(k_{\text{corr}} - 1, r_{\text{corr}} - 1)}} \quad (11)$$

The value ranges from $[0, 1]$, representing the strength of association between the two categorical variables.

D Complete Algorithm

Algorithm 1 provides the complete pseudocode for the StructSynth framework, integrating both the dependency structure discovery and the structure-guided synthesis stages.

E Detailed Experimental Setup

E.1 Datasets

The datasets utilized in this study are summarized in Table 4. They differ in tasks, number of features, and domain. We randomly split each dataset into training and test sets using an 8:2 ratio, yielding $\mathcal{D}_{\text{train-full}}$ and $\mathcal{D}_{\text{test}}$. To simulate data-scarce settings, we further subsample n instances from $\mathcal{D}_{\text{train-full}}$ to form the final training set, $\mathcal{D}_{\text{train}}$. This procedure is repeated ten times with different random seeds (42–51) to ensure robustness.

E.2 Baselines

For all baseline methods, with the exception of CuratedLLM, we utilize the implementations provided in the SynthCity (Qian et al., 2023) library. These models were run using their default hyperparameter configurations to ensure a standardized comparison. For CuratedLLM, we used the official source code provided by the authors, adapting it to our experimental setup. To ensure a fair comparison, it was configured with the same Large Language Model (LLM) settings as our proposed StructSynth, with detailed parameters provided in Table 5.

E.3 Implementation and Hyperparameters

The detailed hyperparameter configurations for the LLMs in StructSynth, the XGBoost model for downstream evaluation, and the Peter-Clark algorithm are provided in Table 5. For the software implementation, we utilize the Langchain library to manage LLM inference and the Causal-Learn library (Zheng et al., 2024) for the PC algorithm.

E.4 Evaluation Metrics

E.4.1 Downstream Model Performance

The Downstream Model Performance metric serves as the primary measure of the synthetic data's practical utility. It evaluates whether the synthetic data, when used to augment the original training set, can improve the performance of a machine learning model on a real, unseen test set. For each generative method, we create an augmented training set, $\mathcal{D}_{\text{aug}}$,

Algorithm 1 StructSynth: Structure-Aware Tabular Data Synthesis

- 1: **Input:** Training data $\mathcal{D}_{\text{train}} = (X_{\text{train}}, A)$, number of synthetic samples s
2: **Output:** Synthetic dataset $\mathcal{D}_{\text{synth}} = (X_{\text{synth}}, A)$

Stage 1: Dependency Structure Discovery*1. Graph Initialization*

- 3: Select a few-shot subset $\mathcal{D}_{\text{few_shot}} \subseteq \mathcal{D}_{\text{train}}$
4: $V_{\text{source}} \leftarrow f_{\text{LLM}}(\pi_{\text{source}}(\mathcal{D}_{\text{few_shot}}))$
5: $G \leftarrow (V_{\text{source}}, \emptyset)$
6: $Q \leftarrow V_{\text{source}}$ ▷ Initialize queue for BFS
7: $V_{\text{visited}} \leftarrow \emptyset$

2. Iterative Expansion, Resolution, and Continuation (BFS)

- 8: **while** $Q \neq \emptyset$ **do**
 a. Expansion and Reasoned Link Generation
 9: $A_i \leftarrow \text{Dequeue}(Q)$
 10: $V_{\text{visited}} \leftarrow V_{\text{visited}} \cup \{A_i\}$
 11: Compute association scores $\mathcal{S}(A_i)$ between A_i and other attributes
 12: $P_i \leftarrow f_{\text{LLM}}(\pi_{\text{generate}}(A_i, G, \mathcal{S}(A_i), \mathcal{D}_{\text{few_shot}}))$
 13: $E_{\text{prop},t} \leftarrow \{(A_i \rightarrow A_j, r_{ij}) \mid (A_j, r_{ij}) \in P_i\}$
 14: $V_{\text{new},t} \leftarrow \{A_j \mid (A_j, _) \in P_i \wedge A_j \notin V\}$
 b. LLM-based Cycle Resolution
 15: $E_{\text{candidate}} \leftarrow E \cup E_{\text{prop},t}$
 16: $\Psi \leftarrow \text{DetectCycles}(V \cup V_{\text{new},t}, E_{\text{candidate}})$
 17: $E_{\text{pruned}} \leftarrow \emptyset$
 18: **for each** cycle $\psi \in \Psi$ **do**
 19: $e_{\text{to_remove}} \leftarrow f_{\text{LLM}}(\pi_{\text{resolve}}(\psi))$
 20: $E_{\text{pruned}} \leftarrow E_{\text{pruned}} \cup \{e_{\text{to_remove}}\}$
 21: **end for**
 c. Continuation of BFS
 22: $G \leftarrow (V \cup V_{\text{new},t}, E_{\text{candidate}} \setminus E_{\text{pruned}})$ ▷ Update graph
 23: Enqueue unvisited nodes from $V_{\text{new},t}$ into Q
24: **end while**
25: **Return** $G = (V, E)$

Stage 2: Structure-Guided Synthesis

- 26: Partition nodes V into topological layers $\mathcal{L} = (L_1, \dots, L_m)$
27: $X_{\text{synth}} \leftarrow \emptyset$
28: **for** $j \leftarrow 1$ to s **do**
 a. Generating Graph-Based Values
 29: $\tilde{x}_{j,V} \leftarrow \{\}$
 30: **for** $i \leftarrow 1$ to m **do**
 31: $L_i \leftarrow \mathcal{L}[i]$
 32: $\tilde{x}_{j,L_i} \leftarrow f_{\text{LLM}}(\pi_{\text{data_gen}}(\tilde{x}_{j,<i}, G[L_i \cup \Gamma^-(L_i)], L_i, \mathcal{D}_{\text{few_shot}}^i))$
 33: $\tilde{x}_{j,V} \leftarrow \tilde{x}_{j,V} \cup \tilde{x}_{j,L_i}$
 34: **end for**
 b. Generating Independent Values
 35: $A_{\text{iso}} \leftarrow A \setminus V$
 36: $\tilde{x}_{j,A_{\text{iso}}} \leftarrow f_{\text{LLM}}(\pi_{\text{data_gen_iso}}(\tilde{x}_{j,V}, A_{\text{iso}}, \mathcal{D}_{\text{few_shot}}^{\text{iso}}))$
 c. Final Assembly
 37: $\tilde{x}_j \leftarrow \text{concat}(\tilde{x}_{j,V}, \tilde{x}_{j,A_{\text{iso}}})$
 38: $X_{\text{synth}} \leftarrow X_{\text{synth}} \cup \{\tilde{x}_j\}$
39: **end for**
40: **Return** $\mathcal{D}_{\text{synth}} = (X_{\text{synth}}, A)$
-

Table 4: Summary of Dataset Characteristics.

Dataset	# Num Feat.	# Cat Feat.	# Classes	Domain	Task	Pre-LLM Knowledge Cu
Adult (Becker and Kohavi, 1996)	4	11	2	Social/Census	Binary	✗
Anxiety (Henderson et al., 2025)	12	7	3	Medical/Health	Multi-Class	✓
Compas (Angwin et al., 2016)	5	3	2	Crime	Binary	✗
Salary (Sajjad, 2025)	3	15	-	Economy	Regression	✓
Obesity (Palechor and de la Hoz Manotas, 2019)	7	8	-	Medical/Health	Regression	✗
Churn (Keramati et al., 2014)	8	4	2	Business	Binary	✗

Table 5: Hyperparameter Settings for Our Experiments.

Group	Hyperparameter Settings
LLMs	frequency_penalty = 0
	presence_penalty = 0
	top_p = 0.95
	temperature = 0.9
	max_tokens = 8000
XGBoost	booster = gbtrees
	n_estimators = 100
	learning_rate = 0.1
	max_depth = 6
	min_child_weight = 1
	gamma = 0
	subsample = 1
	colsample_bytree = 1
PC algorithm	reg_alpha = 0
	reg_lambda = 1
	alpha = 0.05
	indep_test = 'kci'
	stable = True

by combining the original limited training data, $\mathcal{D}_{\text{train}}$, with the generated synthetic data, $\mathcal{D}_{\text{synth}}$:

$$\mathcal{D}_{\text{aug}} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{synth}} \quad (12)$$

A downstream machine learning model (in our experiments, an XGBoost model) is then trained exclusively on this augmented dataset, $\mathcal{D}_{\text{aug}}$, and evaluated on the held-out real test set, $\mathcal{D}_{\text{test}}$. For binary and multi-class classification tasks, we report the Area Under the Receiver Operating Characteristic Curve (AUC). For regression tasks, we report the R-squared (R^2) score.

E.4.2 Privacy Risk Assessment Metric

The Privacy Risk Assessment metric evaluates the extent to which a generative model overfits or memorizes the training data, $\mathcal{D}_{\text{train}}$. The complete set of real data instances is defined as $\mathcal{D}_{\text{real}} = \mathcal{D}_{\text{train}} \cup \mathcal{D}_{\text{test}}$. To ensure an unbiased comparison

where the ideal probability is exactly 0.5, we first downsample the larger of the two sets ($\mathcal{D}_{\text{train}}$ or $\mathcal{D}_{\text{test}}$) to match the size of the smaller one.

For any two data points, x_i and x_j , from the mixed-type dataset, we define a distance function $d(x_i, x_j)$ as the sum of the L1 distance for numerical features and the Hamming distance for categorical features:

$$d(x_i, x_j) = \sum_{k \in \text{numerical}} |x_{i,k} - x_{j,k}| + \sum_{l \in \text{categorical}} \mathbb{I}(x_{i,l} \neq x_{j,l}) \quad (13)$$

For each synthetic sample $\tilde{x} \in \mathcal{D}_{\text{synth}}$, we identify its nearest neighbor (NN) from the entire set of real data instances, $\mathcal{D}_{\text{real}}$:

$$NN(\tilde{x}) = \arg \min_{x \in \mathcal{D}_{\text{real}}} d(\tilde{x}, x) \quad (14)$$

The Privacy Risk score is then calculated as the fraction of synthetic samples whose nearest neighbor is an instance from the training set $\mathcal{D}_{\text{train}}$:

$$\text{PrivacyRisk} = \frac{1}{|\mathcal{D}_{\text{synth}}|} \sum_{\tilde{x} \in \mathcal{D}_{\text{synth}}} \mathbb{I}(NN(\tilde{x}) \in \mathcal{D}_{\text{train}}) \quad (15)$$

A score approaching 0.5 suggests that the generative model has not memorized the training data.

E.4.3 Statistical Fidelity Metric

The Statistical Fidelity metric quantifies how well the synthetic data, $\mathcal{D}_{\text{synth}}$, preserves the pairwise statistical relationships present in the real data, $\mathcal{D}_{\text{real}}$. For each unique pair of columns (C_i, C_j) in the dataset, a difference score, $\Delta(C_i, C_j)$, is calculated. The calculation method varies for the three possible data type combinations:

- **Numerical-Numerical Pairs:** For two numerical columns, the difference is the absolute difference between their Pearson correlation coefficients (ρ) in the real and synthetic datasets:

$$\Delta(C_i, C_j) = |\rho(C_{i,\text{real}}, C_{j,\text{real}}) - \rho(C_{i,\text{synth}}, C_{j,\text{synth}})| \quad (16)$$

- **Categorical-Categorical Pairs:** For two categorical columns, the difference is measured as the Total Variation Distance (TVD) of their joint probability distributions:

$$\Delta(C_i, C_j) = \frac{1}{2} \sum_{u,v} |P_{\text{real}}(C_i = u, C_j = v) - P_{\text{synth}}(C_i = u, C_j = v)| \quad (17)$$

where u and v iterate over all possible categories for columns C_i and C_j , respectively.

- **Numerical-Categorical Pairs:** For a mixed pair of a numerical column C_i and a categorical column C_j , the numerical column is first discretized into a categorical representation, C'_i , by binning its values into $q = 10$ quantiles. The difference is then calculated using the same TVD method as for categorical-categorical pairs:

$$\Delta(C_i, C_j) = \Delta(C'_i, C_j) \quad (18)$$

The overall Statistical Fidelity score is the mean of these difference scores computed over all unique pairs of columns in the dataset:

$$\text{StatisticalFidelity} = \frac{1}{\binom{K}{2}} \sum_{1 \leq i < j \leq K} \Delta(C_i, C_j) \quad (19)$$

where K is the total number of columns.

F Qualitative Graph Analysis

F.1 Structural Fidelity on Adult Dataset

To provide a qualitative illustration of our method’s ability to preserve the core dependencies within the data in the absence of a ground-truth graph, we present a case study on the Adult dataset in Figure 6. This visualization compares three distinct graphs: (a) a reference graph, established by running the PC algorithm on the complete dataset and refining the output with domain expertise; (b) the dependency graph discovered by StructSynth’s learning stage from $\mathcal{D}_{\text{train}}$; and (c) a graph re-discovered from $\mathcal{D}_{\text{synth}}$ generated by StructSynth using the PC algorithm.

A visual comparison reveals a strong alignment between our LLM-discovered graph (Figure 6b) and the reference graph (Figure 6a), particularly around the key *Salary* variable. This demonstrates the effectiveness of our structure learning phase in creating a high-quality dependency graph. More

importantly, the remarkable structural similarity between the re-discovered graph (Figure 6c) and the reference graph serves as powerful evidence of our method’s fidelity. The ability of a standard discovery algorithm to independently recover the essential generative pathways from our synthetic data confirms that StructSynth effectively preserves the underlying dependency structure.

F.2 Structure Discovery Comparisons on Asia Dataset

To provide a concrete illustration of the structural discovery capabilities discussed above, Figure 7 presents a visual comparison of the dependency graphs learned by StructSynth and four representative baselines on the Asia dataset ($n = 100$). The **Ground Truth** graph represents the underlying causal mechanism of the dataset, where, for example, *smoke* causes *lung* (cancer) and *bronc* (bronchitis), which in turn cause *dysp* (dyspnoea).

StructSynth successfully recovers the majority of these key dependencies, including the V-structure at *dysp* and the causal chain from *smoke* to *lung*. The slight deviations are minimal and do not disrupt the global topological ordering, ensuring that the subsequent generation process follows a valid causal path.

In contrast, baseline methods exhibit significant structural distortions:

- **GraDe** tends to produce an overly dense graph with numerous spurious edges (e.g., direct links between *asia* and *dysp*), likely due to its reliance on attention maps that can conflate indirect correlations with direct causation.
- **GOGGLE**, while capturing some dependencies, suffers from edge reversals (e.g., *lung* $\rightarrow$ *smoke*) and misses critical connections (e.g., *bronc* is isolated from *dysp*). This reflects the instability of end-to-end differentiable structure learning in low-data regimes.
- **NoTears** yields a sparse and fragmented graph, failing to identify the relationship between *smoke* and *lung*, which are central to the dataset’s causal logic.
- **FCI**, a constraint-based method, correctly identifies some skeletons but leaves many edges undirected (represented by bi-directed or unoriented edges), failing to provide a clear generative order.

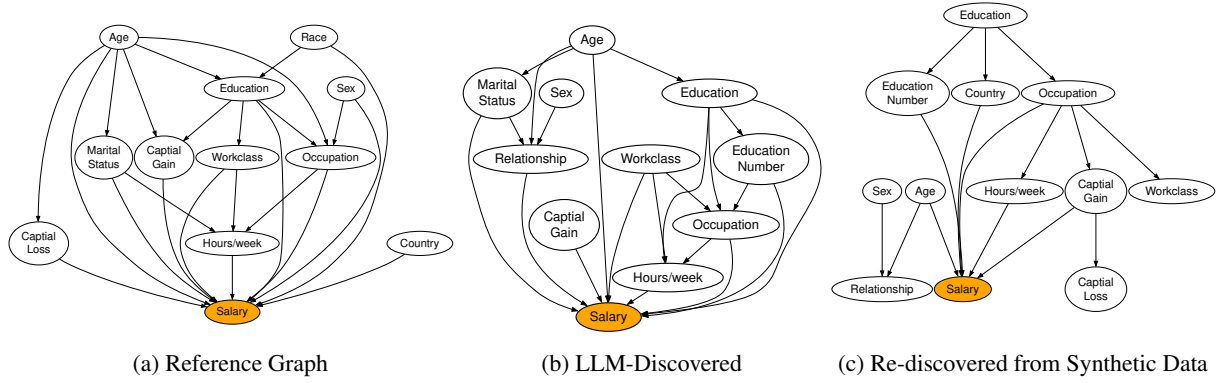


Figure 6: Visual Analysis of Structural Fidelity on the Adult Dataset with *Salary* as the Label Column.

These visual examples corroborate our quantitative findings in Figure 3, confirming that StructSynth’s LLM-guided approach offers superior robustness in discovering interpretable and accurate dependency structures compared to both traditional and deep learning-based alternatives.

G Efficiency Analysis

A practical concern is the additional cost introduced by structure discovery. Table 6 reports the token consumption of CLLM and StructSynth under the **100-shot** setting, averaged over six real-world datasets. Overall, introducing the graph-based structure modeling in StructSynth does *not* lead to a prohibitive cost increase: the average total usage rises from **219.8K** tokens (CLLM total) to **287.9K** tokens (StructSynth combined total), corresponding to an absolute overhead of **+68.1K** tokens (about **+31.0%**). This overhead is relatively stable across datasets (roughly **+19%** to **+42%** in total), suggesting that the additional structure reasoning introduces a consistent but moderate computational budget.

A key observation is that the overhead is **highly skewed toward the input side**. Concretely, StructSynth increases average *input* tokens by **+65.1K (+57.1%)**, while the *output* tokens only increase by **+2.9K (+2.7%)**. This pattern is expected: StructSynth needs additional prompt context to (i) elicit and refine the dependency graph and (ii) condition the synthesis stage on the discovered structure, whereas the corresponding model responses (e.g., edge lists / compact graph representations) are short and the final table synthesis outputs remain comparable in length to CLLM.

Importantly, this “input-dominant” overhead

implies that the **real monetary overhead is often smaller than what the raw total-token increase might suggest**, because in many pricing schemes output tokens are priced higher than input tokens. Taken together, the token analysis indicates that StructSynth achieves structure-aware generation with a **modest and practically manageable** cost increase, largely attributable to additional prompting rather than longer model generations.

H Extended Discussion

In this section, we discuss the broader implications of our results, analyze key design decisions, and contextualize StructSynth within the landscape of structure-aware generative methods.

H.1 Structure-Blind vs. Structure-Aware Generation

A central question raised by our work is whether explicit structural guidance provides a *qualitative*—rather than merely quantitative—advantage over structure-blind LLM generation. To address this, we compare StructSynth against its closest structure-blind counterpart, CLLM, which uses the same LLM with identical hyperparameters but generates data without any structural blueprint.

Our ablation study (Table 3) provides direct evidence for three distinct dimensions of improvement. First, regarding **structural modeling**: CLLM processes features in an arbitrary serialized order, implicitly inducing dependencies from the input sequence. In contrast, StructSynth encodes dependencies as an explicit DAG, ensuring that relationships are *discovered* rather than incidentally induced. Second, regarding **generation strategy**: CLLM generates all features in a single pass without constraints, whereas StructSynth enforces a topological ordering that guarantees each feature

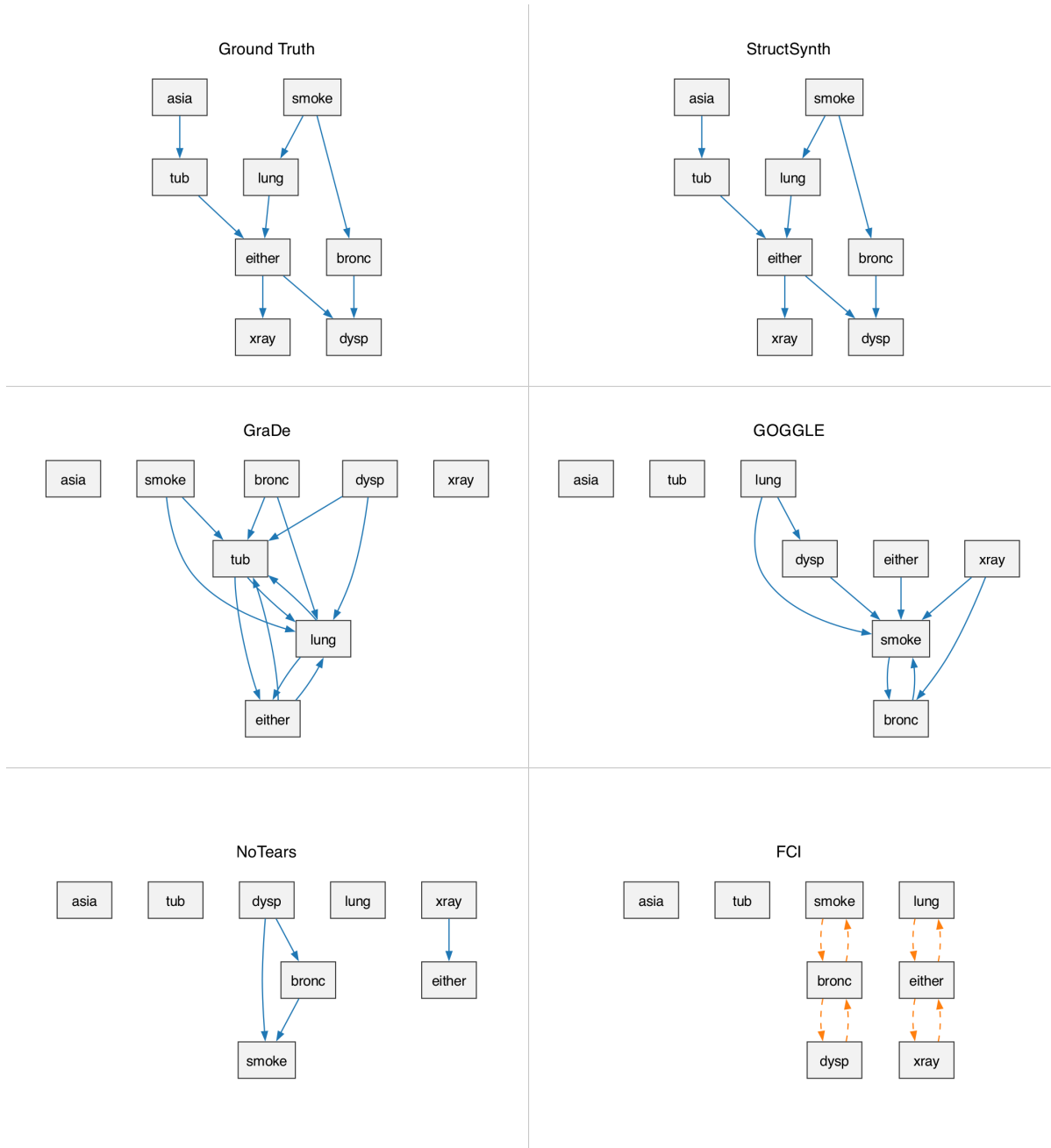


Figure 7: Comparison of discovered dependency structures on the Asia dataset ($n = 100$). **Ground Truth:** The reference DAG from the bnlearn repository. **StructSynth:** Our method closely recovers the ground truth. **GraDe, GOGGLE, NoTears, FCI:** Baseline methods exhibit varying degrees of structural errors, including missing edges, reversed edges, and spurious connections.

is conditioned on its true parent nodes. Removing this ordering alone (*No Topological Order* variant) degrades AUC by 1.1 points, while removing the graph entirely (*No Structure* variant, equivalent to CLLM) causes a 1.6-point drop. Third, regarding **regularization**: the DAG serves as a structural regularizer that constrains the generation space, preventing the LLM from overfitting to spurious patterns in the few available training samples. This

regularization effect is evidenced by StructSynth’s superior privacy preservation (avg. rank 1.33 vs. CLLM’s 3.33), indicating reduced memorization of training records.

These results demonstrate that the advantage of StructSynth is *not* merely the result of adding peripheral components to an LLM pipeline, but stems from a fundamental shift in how dependencies are modeled and enforced during generation.

Table 6: Token usage under 100-shot ($\times 10^3$ tokens). We report input/output tokens for CLLM and StructSynth; overhead compares StructSynth combined totals (graph+table) against CLLM totals.

Method	Component	Adult	Anxiety	Churn	Compas	Obesity	Salary	Avg.
CLLM	Input	40.6	142.2	38.3	26.5	70.7	366.1	114.1
	Output	105.2	175.2	102.6	79.5	13.5	158.6	105.8
	Total	145.8	317.4	140.9	106.0	84.2	524.7	219.8
StructSynth	Graph Input	38.1	89.0	41.4	17.1	43.5	111.1	56.7
	Graph Output	1.5	3.1	2.2	1.0	1.7	3.1	2.1
	Graph total	39.6	92.1	43.6	18.1	45.3	114.3	58.8
	Table Input	52.7	168.3	46.0	32.3	61.1	374.3	122.5
	Table Output	114.3	177.4	102.7	75.9	12.0	157.1	106.6
	Table total	167.0	345.7	148.7	108.2	73.2	531.4	229.0
	Combined total	206.6	437.8	192.3	126.3	118.4	645.7	287.9
	Token overhead of StructSynth vs. CLLM (Combined – Total)							
	Δ Input (K)	50.2	115.1	49.1	22.9	33.9	119.3	65.1
	Δ Output (K)	10.6	5.3	2.3	–2.6	0.2	1.6	2.9
	Δ Total (K)	60.8	120.4	51.4	20.3	34.2	121.0	68.1
	Relative Input (%)	123.6	80.9	128.2	86.4	48.0	32.6	57.1
	Relative Output (%)	10.1	3.0	2.2	–3.3	1.5	1.0	2.7
	Relative Total (%)	41.7	37.9	36.5	19.2	40.6	23.1	31.0

H.2 Understanding the Fidelity-Privacy Trade-off

A nuanced finding in our evaluation is that StructSynth does not achieve the highest Statistical Fidelity score (avg. rank 7.08), despite its strong downstream utility (avg. rank 1.00) and privacy preservation (avg. rank 1.33). We argue that this apparent discrepancy is both expected and desirable, reflecting the inherent tension between fidelity and privacy.

To understand this trade-off, consider GReaT, which achieves the best Statistical Fidelity (avg. rank 3.33). However, its Privacy Risk score of 90.15% reveals severe memorization—the model essentially reproduces training records rather than generating novel samples. This high fidelity is therefore an artifact of overfitting rather than genuine distributional learning. At the other extreme, the Bayesian Sampler in our ablation achieves the best raw fidelity score (48.86) but the lowest AUC (81.17), conclusively demonstrating that **high pairwise fidelity does not guarantee downstream utility**.

This discrepancy can be further understood by examining what Statistical Fidelity actually measures. The metric quantifies the preservation of *pairwise* correlations across all feature pairs. In contrast, StructSynth’s DAG encodes *conditional* dependencies—the probabilistic relationships be-

tween parent and child nodes. These conditional dependencies are what drive downstream model performance, as they capture the generative mechanisms underlying the data. StructSynth optimizes for preserving these conditional structures rather than pairwise statistics, which explains its superior utility despite moderate pairwise fidelity scores.

In summary, StructSynth’s structural blueprint acts as a *principled regularizer*: it constrains the generation process to follow discovered conditional dependencies, which simultaneously prevents memorization (ensuring privacy) and preserves the dependency structure that matters most for downstream tasks (ensuring utility), even at the cost of slightly lower pairwise correlation matching.

H.3 When Does Structural Guidance Help Most?

Our experiments on varying training sample sizes (Figure 4) reveal that StructSynth’s advantage is most pronounced in the low-data regime ($n \leq 50$), where it maintains high AUC while baselines deteriorate significantly. As n increases toward 200, the performance gap narrows, though StructSynth remains competitive.

This pattern has an intuitive explanation. When data is scarce, statistical methods for structure discovery (e.g., PC algorithm, NoTears) suffer from unstable association estimates, leading to

unreliable graphs. In this regime, StructSynth’s LLM-guided discovery provides a powerful complement: the model’s semantic prior knowledge acts as a *structural regularizer*, stabilizing the discovery process by filtering out statistically spurious correlations that would mislead purely data-driven approaches. As more data becomes available, statistical methods become increasingly reliable on their own, diminishing the relative advantage of LLM-guided reasoning.

This analysis also reveals StructSynth’s natural boundary conditions. The method is best suited for scenarios where: (1) the number of training samples is severely limited ($n \leq 100$); (2) features have semantically meaningful names and relationships that an LLM can reason about; and (3) the underlying dependency structure is sparse enough to be well-represented by a DAG. In contrast, for purely numerical datasets with opaque feature names (e.g., anonymized sensor readings), the LLM’s semantic prior provides less value, and the framework may offer diminished advantages over purely statistical approaches.

H.4 Relationship to Causal Inference

It is important to clearly distinguish StructSynth’s dependency graphs from causal graphs in the traditional sense. As established in Appendix B, our DAG serves as a *generative blueprint*—its edges encode statistical and probabilistic dependencies used to organize the synthesis process, not claims of ground-truth causality.

This distinction separates StructSynth from classical causal discovery methods such as the PC algorithm (Spirtes et al., 2000), FCI (Spirtes et al., 2000), or GES (Chickering, 2002), which aim to identify the true causal data-generating mechanism under specific assumptions (e.g., faithfulness, sufficiency). Our objective is fundamentally different: we seek a dependency structure that enables *effective conditional generation*, not one that supports causal interventions or counterfactual reasoning.

Nevertheless, our SHD evaluations on benchmark DAGs (Figure 3) demonstrate that the structures discovered by StructSynth often closely approximate the ground-truth causal graphs. This is not a contradiction: when the true data-generating process follows a DAG, a well-discovered dependency structure will naturally align with the causal structure, even though causal identification is not our explicit goal. This alignment is a desirable side

effect that contributes to the high fidelity of our generated data.

I Detailed Prompts for StructSynth

This section provides a guide to the prompt templates used in the StructSynth framework. A key step in our methodology is the textualization of diverse data structures—including statistical scores, dependency graphs, and tabular data—to serve as inputs for the Large Language Model (LLM). Box 1 shows concrete examples of this process.

The StructSynth pipeline employs a sequence of prompts to complete its tasks. First, the π_{source} prompt initiates graph discovery by identifying initial root nodes. During the iterative graph construction, the $\pi_{\text{generation}}$ prompt proposes new dependency edges with corresponding rationales. If a cycle is detected, the π_{resolve} prompt is used to analyze the conflict and remove the weakest link. Finally, for data synthesis, the $\pi_{\text{data_gen}}$ prompt generates values for dependent features, after which the $\pi_{\text{data_gen_iso}}$ prompt generates values for the remaining isolated features.

Examples of Data Textualization for LLM Prompts

1. Textual Representation of Statistical Evidence (Association Scores):

Association scores for 'Education', scaled 0 to 1.

(Levels: >0.8 Very High, >0.6 High, >0.4 Moderate, >0.2 Low, <=0.2 Very Low)

- Occupation: 0.85 (Very High)
- Salary: 0.72 (High)
- Marital-Status: 0.55 (Moderate)
- Hours-per-week: 0.31 (Low)
- Sex: 0.12 (Very Low)

2. Textual Representation of a Dependency Graph:

Current dependency graph structure with rationales:

- Age -> Marital-Status

Rational: A person's age is a primary factor influencing their marital status.

- Sex -> Relationship:

Rational: Gender often determines the relationship role within a household.

- Country -> Race:

Rational: Native country has a strong statistical correlation with a person's race.

- Race -> Education:

Rational: Statistical data often shows a correlation between a person's race and their level of educational attainment.

3. Textual Representation of Tabular Data (Markdown):

age	education	occupation	sex	hours-per-week	salary
39	Bachelors	Adm-clerical	Male	40	<=50K
50	Bachelors	Exec-managerial	Male	13	<=50K
28	Assoc-acdm	Prof-specialty	Female	45	<=50K

Prompt for Source Nodes Initialization of Tabular Data (π_{source})

You are an expert data analyst. Based on the following information:

Task Description:

Identify the initial "source nodes" for a dependency graph. A source node is a foundational variable that is not caused by any other variable in this dataset.

All Feature Descriptions:

{all_features}

Example Data:

{few_shot_data}

Your Task is to Identify the Source Nodes:

- Your only task is to analyze the provided column descriptions and data.
- Identify the features that are most likely to be **source nodes** (i.e., fundamental attributes that are not effects of other features).
- Provide your answer as a simple list of the identified source node names.

Prompt for Link Generation in Dependency Discovery ($\pi_{\text{generation}}$)

You are an expert data analyst building a dependency graph. Based on the following information:

Task Description:

For the given {current_feature}, your task is to propose its direct successors (effects) from the list of available features. For each proposed dependency, you must provide a brief, evidence-based rationale.

Current Graph State:

{structure_graph}

All Candidate Features:

{all_features}

Statistical Evidence (Association Scores):

{association_scores}

Example Data:

{few_shot_data}

Your Task is to Propose and Justify New Links:

- Your focus is on the feature: {current_feature}.
- Review the **Statistical Evidence**, which shows the strength of association between {current_feature} and other candidate features {all_features}.
- Based on the statistical scores, the example data, and the existing graph, identify which other features are most likely to be **direct effects** of {current_feature}.
- For each dependency you propose, you **must** provide a concise rationale. This rationale should explain why the dependency makes sense (e.g., "Higher Education is strongly correlated with and typically precedes higher Income").
- Do not propose links that would create an obvious logical contradiction with the existing graph structure.
- Format your final output as a list of dependency edges and their corresponding rationales.

Prompt for Cycle Resolution in Dependency Discovery (π_{resolve})

You are a logical reasoning expert tasked with resolving a contradiction in a dependency graph. Based on the following information:

Task Description:

Analyze the provided dependencies and their rationales within the cycle. Identify the single dependency with the weakest or least plausible rationale and recommend it for removal to break the cycle.

Conflicting Cycle with Rationales:

{cycle_with_rationales}

Your Task is to Identify the Weakest Link:

- A logical contradiction (a cycle) has been detected in the graph, detailed in **Conflicting Cycle with Rationales**.
- Your task is to carefully evaluate the rationale provided for each edge in this cycle.
- Based on your analysis of the rationales and any relevant context from the column descriptions or data, identify the **single weakest link**.
- The weakest link is the dependency whose rationale is the least plausible, least supported, or most likely to be spurious.
- Provide your final answer as the single dependency edge that should be removed to resolve the contradiction.

Prompt for Structure-Guided Data Synthesis ($\pi_{\text{data_gen}}$)

You are a helpful AI assistant that generates realistic tabular data based on structural dependencies.

Task Description:

Generate a realistic synthetic table containing exactly the columns listed in {features_to_generate}. This generation must be conditioned on the provided values of their parent features.

Conditioning Data (Current Results):

{current_result}

Relevant Dependency Structure:

{related_structure_graph}

Example Data:

{related_few_shot_data}

Your Task is to Generate the Synthetic Data:

- Your primary task is to generate realistic data for the features listed in {features_to_generate}.
- This generation must be **conditioned** on the data provided in **Conditioning Data (Current Results)**. These are the values of the parent nodes that influence the features you are generating.
- The **Relevant Dependency Structure** shows you exactly how the conditioning features relate to the features you need to generate.
- Use the **Example Data** to understand the format, range, and statistical properties of the real data.
- The generated table should contain columns for exactly the features listed in {features_to_generate}.
- Ensure the generated values are plausible and respect the learned dependencies.

Prompt for Independent Feature Synthesis ($\pi_{\text{data_gen_iso}}$)

You are a helpful AI assistant that completes tabular data records by generating values for remaining features.

Task Description:

The core, structurally-dependent features of a dataset have already been generated. Your task is to generate plausible values for the remaining isolated features, ensuring they are statistically consistent with the core features.

Conditioning Data (Generated Graph-Based Features):

{graph_based_values}

Features to Generate (Isolated Features):

{isolated_features_to_generate}

Example Data:

{related_few_shot_data}

Your Task is to Generate the Independent Values:

- Your primary task is to generate realistic data for the features listed in {isolated_features_to_generate}.
- This generation must be **conditioned** on the complete set of core features provided in **Conditioning Data**.
- The features you are generating do not have direct parent-child dependencies in the learned graph, but their values should still be plausible and consistent in the context of the entire data record.
- Use the **Example Data** to understand the typical format and statistical properties of these isolated features.
- The generated table should contain columns for exactly the features listed in {isolated_features_to_generate}.